

How it was built, and how to run it yourself

The backend, the databases, Docker, the AI service, the chatbot, the phone app and the admin website: how each was made, and how to start, check, change and repair it without an AI assistant.

AI-Driven Mobile Health Platform for Prostate Cancer Diagnosis (Zambia)

Final-year research prototype · made on 1 October 2026 · code pictures from commit 9104c32 with uncommitted changes

Not a medical device. All data shown is synthetic.

This manual explains every part of the system (the backend, the databases, Docker, the AI service, the chatbot, the phone app and the admin website): how each was made, and how to start, stop, check, change and repair it **without an AI assistant**.

- **Who it is for:** the project owner, and anyone who takes the project over.
- **What you need to know already:** how to open PowerShell and how to open a file in an editor. Everything else is written out.
- **Where commands are run:** in PowerShell, from the project folder `D:\Final Year Project\pca-mhealth`, unless a step says otherwise.
- **The pictures are made from the project itself** (section 17). The code pictures show the real files, each with its file name and line numbers, so you can open the same place yourself; the cover of the PDF says from which commit they were taken. The terminal pictures show the real output of the commands. The phone and admin pictures use synthetic data.
- **All data is synthetic.** The system is a research prototype, not a medical device.

Contents

1. [The system on one page](#)
2. [What is installed on this PC](#)
3. [Starting and stopping](#)
4. [The settings file \(.env\)](#)
5. [The backend](#)
6. [The databases](#)
7. [Docker](#)
8. [The AI service](#)
9. [The chatbot, end to end](#)
10. [The phone app](#)
11. [The admin website](#)
12. [Tests and the quality gate](#)
13. [Git and GitHub](#)
14. [When something goes wrong](#)
15. [Before real patients](#)
16. [Where to read more](#)
17. [Updating this manual](#)

1. The system on one page

PCa mHealth helps clinics in Zambia screen for prostate cancer. Clinicians register patients and record screenings on a phone, also without internet. Pathologists review slides. Patients see their own results, read or listen to health information, and ask an assistant questions. Administrators manage accounts on a website.

The pieces

Piece	What it does	Built with	Folder	Address on this PC
Phone app	Screens for patients, clinicians and pathologists; works offline	Flutter 3.35 (Dart)	1-presentation-layer/mobile-app	runs on the emulator or a phone
Admin website	Accounts, roles, patient links, audit log, dashboard	React 19 + Vite 8 (TypeScript)	1-presentation-layer/admin-panel-web	http://localhost:5173
Backend (the API)	Every rule of the system: sign-in, permissions, patients, consent, uploads, sync, chat, audit	NestJS 11 (TypeScript) on Node 24	3-application-logic/backend	http://localhost:3000
AI service	The AI models (labelled mock models for now) and the chatbot's search	FastAPI (Python 3.14)	4-ai-intelligence-layer/ai-services	http://127.0.0.1:8000
PostgreSQL	Accounts, patients, screenings, consents, audit log	PostgreSQL 18, installed on the PC	schema in 3-application-logic/backend/prisma , history in 5-data-persistence/postgresql/migrations	localhost:5432
MongoDB	AI reports, chats, scan details	MongoDB 8.3, installed on the PC	3-application-logic/backend/src/persistence/mongo	localhost:27017
Redis, MinIO, Qdrant	Shared rate limits; S3 file storage; vector search (planned)	Docker containers	6-infrastructure/docker	6379; 9000 and 9001; 6333

How a request travels

```

Phone app / admin website
    |
    | HTTPS in production; http://localhost:3000 on this PC
    v
Backend (NestJS)
  1. rate limit per network address
  2. who are you? (JWT access token)
  3. rate limit per account
  4. may you do this? (role permissions)
  5. is the request well-formed? (validation)
  6. the service does the work and writes an audit entry
      |           |           |
      v           v           v
PostgreSQL 18   MongoDB 8.3   AI service (FastAPI)
(records)       (documents)   (models, chatbot search)

```

The phone app never talks to a database or to the AI service. Everything goes through the backend, so every rule is enforced in one place.

Why these choices (each has a decision record in [docs/decisions/](#)): - Flutter, because one codebase gives Android and iPhone apps with good offline support (ADR-001). - NestJS with PostgreSQL and MongoDB, and a separate Python service for AI, because AI libraries live in Python while the rest benefits from TypeScript's type checks (ADR-003). - A website for administrators instead of phone screens, because administration is desk work (ADR-005). - A chatbot that only answers from reviewed documents, because a health assistant must never invent medical text (ADR-009, ADR-010).

2. What is installed on this PC

Checked on 30 September 2026:

Tool	Version	Used for	How to check
Node.js / npm	24.15.0 / 11.12.1	backend, admin website, scripts	<code>node -v</code>
Python	3.14.5	AI service	<code>python --version</code>
Flutter / Dart SDK	3.35.2 / 3.9	phone app	<code>flutter --version</code>
Android Studio + SDK	emulator Galaxy_S9_Plus_API_29	running the app	Android Studio > Device Manager
PostgreSQL	18.6 (service postgresql-x64-18), with pgAdmin 4	main database	<code>& "C:\Program Files\PostgreSQL\18\bin\psql.exe" --version</code>

Tool	Version	Used for	How to check
MongoDB	8.3.2 (service MongoDB), with Compass	document database	Services (services.msc)
Docker Desktop	29.8.0	Redis, MinIO, Qdrant	<code>docker --version</code>
Git	2.51	version control	<code>git --version</code>
Google Chrome		admin website	

On a new PC, install the same tools, then follow [section 3.5](#).

3. Starting and stopping

3.1 Start everything with one command

```
cd "D:\Final Year Project\pca-mhealth"
powershell -ExecutionPolicy Bypass -File 6-infrastructure\scripts\dev-up.ps1
```

The script does this, in order:

1. Checks that the `.env` settings file exists.
2. Checks that the two database services (**postgresql-x64-18** and **MongoDB**) are running, and starts them if not.
3. Starts Docker Desktop if needed, then Redis, MinIO and Qdrant.
4. Stops an old backend if one is still running.
5. Applies any new database changes (`prisma migrate deploy`) and makes sure the synthetic demo data exists (`db:seed`).
6. Builds the backend and starts it in its own window.
7. Waits until the backend answers.
8. Starts the admin website in its own window.
9. Starts the AI service in its own window.

Keep the three windows open. Closing a window stops that piece.

This is the part of the script that looks after the databases:

```
6-infrastructure/scripts/dev-up.ps1 lines 30-43

30 # The databases installed on this PC. Both services start with Windows; this
31 # starts them if they were stopped (that needs an administrator PowerShell).
32 if (-not $DockerDatabases) {
33     foreach ($name in @('postgresql-x64-18', 'MongoDB')) {
34         $service = Get-Service -Name $name -ErrorAction SilentlyContinue
35         if (-not $service) { Fail "The $name service is not installed. See docs\local-databases.md, or add -
DockerDatabases." }
36         if ($service.Status -ne 'Running') {
37             Say "Starting the $name service..."
38             try { Start-Service -Name $name } catch {
39                 Fail "Could not start $name. Open PowerShell as administrator and run: Start-Service $name"
40             }
41         }
42     }
43 }
```

3.2 Check that everything is up

Open this	You should see
http://localhost:3000/api/v1/health	"status":"ok"
http://127.0.0.1:8000/v1/health	"status":"ok"; chat_writer says quotes (or claude:... when Claude is switched on); chat_retrieval says keywords+meaning (MedCPT) a few seconds after start-up (section 9.4)
http://localhost:5173	the admin sign-in page
http://localhost:3000/api/docs	the API explorer, listing every route

```
Is everything up?

PS D:\Final Year Project\pca-mhealth> curl.exe -s http://localhost:3000/api/v1/health
{"status":"ok","service":"pca-mhealth-backend","timestamp":"2026-10-01T11:56:12.009Z"}

PS D:\Final Year Project\pca-mhealth> curl.exe -s http://127.0.0.1:8000/v1/health
{"status":"ok","service":"pca-mhealth-ai-services","mode":"development-
mock","clinical_models_loaded":false,"chat_writer":"quotes","chat_retrieval":"keywords+meaning (MedCPT)"}

```

3.3 Stop everything

```
powershell -ExecutionPolicy Bypass -File 6-infrastructure\scripts\dev-down.ps1
```

It stops the backend, the admin website, the AI service and the Docker containers. **No data is lost.** PostgreSQL and MongoDB keep running as Windows services, which is normal.

3.4 Start each piece by hand

Use this when you want to see one piece's messages, or when the script fails.

Databases (usually already running):

```
Get-Service postgresql-x64-18, MongoDB          # both should say Running
Start-Service postgresql-x64-18                 # needs PowerShell "Run as administrator"
Start-Service MongoDB
```

Docker services:

```
docker compose -f 6-infrastructure\docker\docker-compose.yml --env-file .env up -d
```

Backend:

```
cd 3-application-logic\backend
npm ci                    # only the first time, or after package.json changed
npx prisma migrate deploy # apply database changes
npm run db:seed           # synthetic demo data (safe to repeat)
npm run build
npm run start:prod        # or: npm run start:dev (restarts when you save a file)
```

AI service:

```
cd 4-ai-intelligence-layer\ai-services
py -3 -m venv .venv                # only the first time
.venv\Scripts\python -m pip install -e ".[dev]" # only the first time
.venv\Scripts\python -m uvicorn app.main:app --host 127.0.0.1 --port 8000
```

Admin website:

```
cd 1-presentation-layer\admin-panel-web
npm ci          # only the first time
npm run dev     # then open http://localhost:5173
```

Phone app: see [section 10.2](#).

3.5 Setting up on a new PC

1. Install the tools in section 2.
2. Get the code: `git clone https://github.com/deedeestephen/final-year-project.git`
3. Create the settings file with fresh random secrets: `node 6-infrastructure\scripts\gen-keys.mjs --init-env`
4. Create the database login and database: follow steps 1 and 3 of [local-databases.md](#), and put the same passwords in `.env`.
5. Install the AI service (the two "only the first time" lines above).
6. Run `dev-up.ps1`. It installs the other packages, creates the tables and adds the demo data.

3.6 Signing in

The seed creates four synthetic demo accounts:

Account	Role	Where it signs in
admin@demo.pca-mhealth.test	Administrator	admin website
clinician@demo.pca-mhealth.test	Clinician	phone app
pathologist@demo.pca-mhealth.test	Pathologist	phone app
patient@demo.pca-mhealth.test	Patient	phone app

- **The password** of all four is the value of `SEED_DEMO_PASSWORD` in `.env`.
- The first time an account signs in, it may be asked to choose its own password (12 or more characters).
- **Forgot it, or locked out?** Five wrong tries lock an account for 15 minutes. This command puts all four back to the password in `.env` and removes the lock:

```
powershell -ExecutionPolicy Bypass -File 6-infrastructure\scripts\dev-up.ps1 -ResetDemoPasswords
```

4. The settings file (.env)

`.env` sits in the project folder. It holds every setting and every secret: database passwords, the keys that sign sessions, the key that encrypts personal fields.

- **It is never committed to Git.** `.gitignore` excludes it, and a secret scanner checks every commit.
- `.env.example` **is not the settings file.** It is a template with empty or placeholder values, which *is* in Git, so a new developer knows which settings exist.
- **How it was made:** `node 6-infrastructure\scripts\gen-keys.mjs --init-env` copies the template and fills every secret with random values.
- **After changing `.env`, restart the piece that reads it** (the backend or the AI service).
- **A value that contains `$` must be in single quotes**, for example `SEED_DEMO_PASSWORD='$example'`. Otherwise Docker reads `$example` as a variable and prints a warning.

```
.env.example
lines 13-33

13 # --- PostgreSQL ---
14 # The project uses the PostgreSQL 18 installed on the PC (Windows service, port 5432);
15 # docs/local-databases.md shows how the pca login and database are created.
16 POSTGRES_USER=pca
17 POSTGRES_PASSWORD=change-me
18 POSTGRES_DB=pca_mhealth
19 DATABASE_URL=postgresql://pca:change-me@localhost:5432/pca_mhealth?schema=public
20 # The admin (postgres) password chosen when PostgreSQL was installed. Only the set-up
21 # steps in docs/local-databases.md use it; the app never does. Leave empty if not needed.
22 LOCAL_POSTGRES_ADMIN_PASSWORD=
23 # Only for the optional Docker database (dev-up.ps1 -DockerDatabases): its host port.
24 # Then DATABASE_URL must use this port too.
25 POSTGRES_PORT=5433
26
27 # --- MongoDB ---
28 # The project uses the MongoDB installed on the PC (Windows service, port 27017).
29 MONGO_USER=pca
30 MONGO_PASSWORD=change-me
31 MONGO_URL=mongodb://pca:change-me@localhost:27017/pca_mhealth?authSource=admin
32 # Only for the optional Docker database (dev-up.ps1 -DockerDatabases); then MONGO_URL uses it.
33 MONGO_PORT=27018
```

The settings in groups

Group	Settings	Meaning
Backend	PORT , CORS_ORIGINS , RATE_LIMIT_MAX , USER_RATE_LIMIT_MAX	port 3000; which website may call the API; 600 requests a minute per address, 120 per account
PostgreSQL	DATABASE_URL , POSTGRES_USER , POSTGRES_PASSWORD , LOCAL_POSTGRES_ADMIN_PASSWORD	where the main database is; the app's login; the admin login, used only for set-up
MongoDB	MONGO_URL , MONGO_USER , MONGO_PASSWORD	where the document database is
Redis	REDIS_URL , REDIS_PASSWORD	shared rate-limit counters; without it each backend counts alone
Files	STORAGE_DRIVER , LOCAL_STORAGE_ROOT , S3_* , MINIO_*	local (now): files go to backend/var/objects ; s3 : files go to MinIO
Sign-in	JWT_PRIVATE_KEY_BASE64 , JWT_PUBLIC_KEY_BASE64	the key pair that signs sessions
Encryption	FIELD_ENCRYPTION_KEY_BASE64 , FIELD_HMAC_KEY_BASE64	encrypt names, phone and NRC numbers. If these are lost, those fields cannot be read again: back up .env safely
AI	AI_SERVICE_URL , AI_SERVICE_TOKEN , ANTHROPIC_API_KEY , CHAT_LLM_MODEL , CHAT_LLM_DAILY_LIMIT	where the AI service is; the shared token; Claude is off while the key is empty

Group	Settings	Meaning
Chatbot search	CHAT_RETRIEVAL , CHAT_MODELS_DIR , CHAT_MEANING_SLOTS , CHAT_MEANING_WAIT_MS	auto : the meaning search is used when its models are downloaded (keywords switches it off); where the models are; how many questions it handles at once, and how long one waits before keywords alone answer (section 9.4)
Demo	SEED_DEMO_PASSWORD	the password of the four demo accounts

5. The backend

5.1 What it is and how it is organised

The backend is a NestJS application: a web server that answers requests under `http://localhost:3000/api/v1/...`. Its source is in `3-application-logic/backend/src`, in three folders that match the architecture:

Folder	What is inside
gateway/	What every request passes through: sign-in check, permissions, rate limits, validation, error format, logging, upload checks
services/	One folder per feature: <code>auth</code> , <code>users</code> , <code>patients</code> , <code>clinical</code> , <code>imaging</code> , <code>ai</code> , <code>chatbot</code> , <code>sync</code> , <code>notifications</code> , <code>fhir</code> , <code>admin</code> , <code>audit</code>
persistence/	Talking to stores: <code>database</code> (PostgreSQL through Prisma, and MongoDB), <code>storage</code> (files), <code>crypto</code> (field encryption), <code>mongo</code> (collection rules), <code>seed.ts</code> (demo data)

Each feature folder has the same three kinds of file: - a **controller** (`*.controller.ts`): the routes, and the permission each one needs; - a **service** (`*.service.ts`): the rules and the database work; - a **DTO** file (`*.dto.ts`): the exact shape of what may be sent and what comes back.

5.2 How it starts

`main.ts` is the first file that runs. It reads `.env`, builds the application, applies the security settings and starts listening on the port.

```

1 import { NestFactory } from '@nestjs/core';
2 import type { NestExpressApplication } from '@nestjs/platform-express';
3 import { Logger } from 'nestjs-pino';
4 import { AppModule } from './app.module';
5 import { APP_CONFIG, type AppConfig } from './config/app-config';
6 import { configureApp } from './gateway/configure-app';
7 import { setupOpenApi } from './gateway/openapi';
8 import { loadRootEnv } from './config/repo-root';
9
10 async function bootstrap() {
11   // Local development reads the repository-root .env; deployments inject variables.
12   loadRootEnv();
13
14   const app = await NestFactory.create<NestExpressApplication>(AppModule, {
15     bufferLogs: true,
16     bodyParser: false, // re-enabled with explicit size limits in configureApp
17   });
18   app.useLogger(app.get(Logger));
19   const config = app.get<AppConfig>(APP_CONFIG);
20   configureApp(app, config);
21   if (config.apiDocsEnabled) setupOpenApi(app);
22   await app.listen(config.port);
23 }
24 void bootstrap();

```

5.3 Four guards on every request

`app.module.ts` registers four guards that run on **every** route, in this order. A route is closed unless the code says otherwise ("deny by default").

```

86 providers: [
87   // Gateway concerns applied to every route, in this order, in tests exactly as in production:
88   // address rate limit -> authenticate (deny by default) -> per-account
89   // rate limit -> authorise (RBAC).
90   { provide: APP_GUARD, useClass: AppThrottlerGuard },
91   { provide: APP_GUARD, useClass: JwtAuthGuard },
92   { provide: APP_GUARD, useClass: UserRateLimitGuard },
93   { provide: APP_GUARD, useClass: PermissionsGuard },
94   { provide: APP_FILTER, useClass: AllExceptionsFilter },
95   { provide: APP_PIPE, useFactory: createValidationPipe },
96 ],
97 })
98 export class AppModule {}

```

1. **AppThrottlerGuard**: at most 600 requests a minute from one network address.
2. **JwtAuthGuard**: the request must carry a valid session token. Only routes marked `@Public()` (sign-in, health) skip this.
3. **UserRateLimitGuard**: at most 120 requests a minute per account.
4. **PermissionsGuard**: the account's roles must include the permission the route asks for.

The permissions guard compares what the route needs with what the user has, and writes an audit entry when it refuses:

```

29  async canActivate(context: ExecutionContext): Promise<boolean> {
30    const targets = [context.getHandler(), context.getClass()];
31    if (this.reflector.getAllAndOverride<boolean>(IS_PUBLIC, targets))
32      return true;
33
34    const required =
35      this.reflector.getAllAndMerge<PermissionCode[]>(
36        REQUIRED_PERMISSIONS,
37        targets,
38      ) ?? [];
39    if (required.length === 0) return true;
40
41    const req = context.switchToHttp().getRequest<AuthenticatedRequest>();
42    const user = req.user;
43    const missing = required.filter((p) => !user?.permissions.includes(p));
44    if (missing.length === 0) return true;

```

- **Roles:** PATIENT, CLINICIAN, PATHOLOGIST, ADMIN.
- **Permissions** are codes such as patient:read or user:manage. Roles hold permissions, and users hold roles.
- **Every route and who may use it** is listed in [access-matrix.md](#) (73 routes). That file is generated from the code with `npm run access:matrix`, and the quality gate fails if it is out of date.

5.4 One feature from top to bottom: deleting an account

The **route** says which address it answers, which permission it needs, and what it returns:

```

103  /** Only accounts without clinical history; the others can be disabled. */
104  @Delete('/:id')
105  @HttpCode(HttpStatus.NO_CONTENT)
106  @RequirePermissions('user:manage')
107  @ApiNoContentResponse({ description: 'The account was deleted' })
108  @ApiConflictResponse({
109    description:
110      'HAS_CLINICAL_HISTORY: the account has clinical history; disable it instead',
111  })
112  remove(
113    @Param('id', new ParseUUIDPipe()) id: string,
114    @CurrentUser() actor: AuthenticatedUser,
115    @Ctx() ctx: RequestContext,
116  ): Promise<void> {
117    return this.users.delete(id, actor, ctx);
118  }
119 }

```

The **service** holds the rules. It refuses the administrator's own account and any account named in the clinical record. Otherwise it deletes the account and writes the audit entry in one database transaction (both happen or neither does). Last, it removes the account's chats from MongoDB.

```

373   async delete(
374     id: string,
375     actor: AuthenticatedUser,
376     ctx: RequestContext,
377   ): Promise<void> {
378     if (id === actor.id) {
379       throw new BadRequestException({
380         code: 'SELF_DELETE',
381         message: 'You cannot delete your own account',
382       });
383     }
384     const existing = await this.prisma.user.findUnique({
385       where: { id },
386       include: { ...WITH_ROLES, patientProfile: { select: { id: true } } },
387     });
388     if (!existing)
389       throw new NotFoundException({
390         code: 'NOT_FOUND',
391         message: 'User not found',
392       });
393     const history = await this.clinicalHistory(id);
394     if (Object.values(history).some((n) => n > 0)) {
395       throw new ConflictException({
396         code: 'HAS_CLINICAL_HISTORY',
397         message:
398           'This account has clinical history, so it cannot be deleted. Disable it instead: it can no longer
sign in, and the history stays traceable.',
399         details: history,
400       });
401     }
402     await this.prisma.$transaction(async (tx) => {
403       await tx.user.delete({ where: { id } });
404       await this.audit.record(
405         {
406           action: 'user.deleted',
407           entityType: 'user',
408           entityId: id,
409           outcome: 'SUCCESS',
410           actorUserId: actor.id,
411           actorRole: actor.roles.join(','),
412           details: {
413             roles: existing.roles.map((r) => r.role.name).sort(),
414             wasLinkedToPatient: existing.patientProfile !== null,
415             synthetic: existing.isSynthetic,
416           },
417           ...ctx,
418         },
419         tx,
420       );
421     });
422     await (
423       await this.mongo.db()
424     )
425       .collection(CHAT_COLLECTION)
426       .deleteMany({ userId: id });
427   }
428 }

```

Every feature follows this pattern: controller → service → Prisma (database) → audit.

5.5 Security that is built in

Protection	How
Passwords	Stored only as Argon2id hashes (64 MB memory, 3 passes). Nobody, including an administrator, can read a password. At least 12 characters
Sessions	A short access token (15 minutes), signed with an Ed25519 key pair, plus a refresh token (14 days) that is replaced on every use. A stolen, reused refresh token ends the whole session family
Lockout	5 wrong passwords lock the account for 15 minutes
Personal fields	Names, phone and NRC numbers are encrypted in the database (AES-256-GCM). The NRC can still be matched exactly through a keyed hash
Audit log	Every important action is recorded. The table only accepts new rows, and each row carries a hash of the row before it, so any later change is detected
Input	Every request body is validated against its DTO; unknown fields are rejected
Uploads	Size limits, file-type checks by content, DICOM header checks

The audit chain is enforced by the database itself, not only by the application:

```
5-data-persistence/postgresql/migrations/20260923203337_constraints_and_audit/migration.sql lines 75-92

75 CREATE OR REPLACE FUNCTION audit_logs_before_insert()
76 RETURNS trigger LANGUAGE plpgsql AS $$
77 DECLARE
78     v_prev text;
79 BEGIN
80     PERFORM pg_advisory_xact_lock(hashtext('audit_logs_chain'));
81     NEW.seq := nextval(pg_get_serial_sequence('audit_logs', 'seq'));
82     NEW.occurred_at := date_trunc('milliseconds', coalesce(NEW.occurred_at, now()));
83     SELECT row_hash INTO v_prev FROM audit_logs ORDER BY seq DESC LIMIT 1;
84     NEW.prev_hash := coalesce(v_prev, repeat('0', 64));
85     NEW.row_hash := audit_logs_row_hash(NEW.prev_hash, NEW);
86     RETURN NEW;
87 END;
88 $$;
89
90 CREATE TRIGGER audit_logs_chain
91 BEFORE INSERT ON audit_logs
92 FOR EACH ROW EXECUTE FUNCTION audit_logs_before_insert();
```

On the admin website, **Audit log** > **Check integrity** re-computes the whole chain.

5.6 Backend commands

Run these in `3-application-logic\backend`:

Task	Command
Install packages	<code>npm ci</code>
Build	<code>npm run build</code>

Task	Command
Start (built code)	<code>npm run start:prod</code>
Start and restart on every save	<code>npm run start:dev</code>
Unit tests	<code>npm test</code>
End-to-end tests (no database needed)	<code>npm run test:e2e</code>
Database tests (need PostgreSQL, MongoDB and Docker's MinIO)	<code>npm run test:db</code>
Whole workflows on the live system	<code>npm run test:workflows</code>
Format and lint	<code>npx prettier --write src test</code> and <code>npm run lint</code>
Rebuild the API description	<code>npm run openapi:export</code>
Rebuild the route list	<code>npm run access:matrix</code>
Reset the demo passwords	<code>npm run db:reset-demo</code>

5.7 The API explorer

`http://localhost:3000/api/docs` lists every route with its inputs and outputs. To try one: sign in through `POST /api/v1/auth/login`, copy the `accessToken`, click **Authorize** and paste it. The same description is saved in `2-api-gateway/openapi/openapi.json`.

5.8 Adding a new route

1. Pick the feature folder in `src/services/` (or create one with a module, controller and service).
2. Describe the input and output in the `*.dto.ts` file.
3. Add the method to the service. Record an audit entry for anything that changes data.
4. Add the route to the controller with `@RequirePermissions('...')`.
5. Write a test: a unit test next to the file (`*.spec.ts`), or a database test in `test/db/`.
6. Run `npm run openapi:export` and `npm run access:matrix`.
7. Run the gate: `powershell -ExecutionPolicy Bypass -File 6-infrastructure\scripts\quality-gate.ps1 backend`.

6. The databases

6.1 What is stored where

Data	Where	Why there
Accounts, roles, patients, screenings, consents, scans and slides (the records), AI jobs, notifications, sync history, audit log	PostgreSQL 18, database <code>pca_mhealth</code> , 20 tables	Structured records with strict rules and links between them

Data	Where	Why there
AI reports, AI logs, scan technical details, chatbot conversations	MongoDB 8.3, database pca_mhealth , 4 collections	Documents whose shape varies
Uploaded scan and slide files	the folder 3-application-logic/backend/var/objects	Large files do not belong in a database
Rate-limit counters	Redis (Docker)	Fast, shared, temporary

```

psql: the tables in PostgreSQL 18

PS D:\Final Year Project\pca-mhealth> & "C:\Program Files\PostgreSQL\18\bin\psql.exe" -h localhost -U pca -d
pca_mhealth -c "\dt"

      List of tables
Schema |          Name          | Type  | Owner
-----+-----+-----+-----
public | _prisma_migrations     | table | pca
public | ai_jobs                 | table | pca
public | ai_models               | table | pca
public | audit_logs              | table | pca
public | clinical_records        | table | pca
public | consents                | table | pca
public | explainability_artifacts | table | pca
public | facilities              | table | pca
public | histopathology_specimens | table | pca
public | imaging_studies         | table | pca
public | notifications           | table | pca
public | password_reset_tokens   | table | pca
public | patients                | table | pca
public | permissions             | table | pca
public | refresh_tokens          | table | pca
public | role_permissions        | table | pca
public | roles                   | table | pca
public | sync_operations         | table | pca
public | user_roles              | table | pca
public | users                   | table | pca
(20 rows)

```

6.2 PostgreSQL: how the tables were made

The tables are **not** created by hand. They are described in one file, 3-application-logic/backend/prisma/schema.prisma . This is the part that describes a user account:

```

24 model User {
25   id          String      @id @default(uuid()) @db.Uuid
26   email       String      @unique // stored lower-cased by the application
27   passwordHash String      @map("password_hash")
28   displayName  String      @map("display_name")
29   status       UserStatus @default(ACTIVE)
30   failedLoginCount Int      @default(0) @map("failed_login_count")
31   lockedUntil  DateTime?   @map("locked_until") @db.Timestampz(3)
32   lastLoginAt  DateTime?   @map("last_login_at") @db.Timestampz(3)
33   mustChangePassword Boolean @default(false) @map("must_change_password")
34   facilityId   String?     @map("facility_id") @db.Uuid
35   isSynthetic  Boolean      @default(false) @map("is_synthetic")
36   /// Patient self-registration: contact phone and identity document, encrypted
37   /// (AES-256-GCM). The HMAC allows exact matching to a clinic record without
38   /// storing the number in clear.
39   phoneEnc     Bytes?       @map("phone_enc")
40   idDocumentType String?    @map("id_document_type") // NRC | PASSPORT
41   idNumberEnc  Bytes?       @map("id_number_enc")
42   idNumberHmac String?     @unique @map("id_number_hmac")
43   createdAt    DateTime     @default(now()) @map("created_at") @db.Timestampz(3)
44   updatedAt    DateTime     @updatedAt @map("updated_at") @db.Timestampz(3)
45
46   facility      Facility?      @relation(fields: [facilityId], references: [id])
47   roles         UserRole[]
48   refreshTokens RefreshToken[]
49   passwordResetTokens PasswordResetToken[]
50   notifications Notification[]
51   patientProfile Patient?      @relation("PatientAccount")
52
53   @@index([facilityId])

```

From that file, the tool **Prisma** generates: - **migrations**: SQL files in `5-data-persistence/postgresql/migrations/`, one folder per change, in date order. They are the history of the database, and they are in Git. - **the Prisma client**: the typed code the backend uses to read and write (`this.prisma.user.findUnique(...)`).

To change the database:

1. Edit `schema.prisma`.
2. Create the migration and apply it to your database: `powershell cd 3-application-logic\backend npm run db:migrate:dev -- --name describe_the_change`
3. Read the new SQL file in the migrations folder. Special rules (such as the audit triggers) are written into it by hand.
4. Commit the schema and the migration together.

To apply existing migrations (done by `dev-up.ps1`): `npx prisma migrate deploy`. It only ever adds what is missing.

To check:

```

Prisma: is the database schema up to date?

PS D:\Final Year Project\pca-mhealth\3-application-logic\backend> npx prisma migrate status
Loaded Prisma config from prisma.config.ts.

Prisma config detected, skipping environment variable loading.
Prisma schema loaded from prisma\schemas\prisma
Datasource "db": PostgreSQL database "pca_mhealth", schema "public" at "localhost:5432"

6 migrations found in prisma/migrations

Database schema is up to date!

```

Never run `prisma migrate reset`. It deletes every table and all data.

6.3 MongoDB: collections with rules

MongoDB would accept anything, so each collection has a validator: a rule the server checks on every write. They are defined in `src/persistence/mongo/collections.ts` and applied with:

```
npm run db:mongo:migrate
```

Collection	Holds
<code>ai_reports</code>	the result of each AI analysis, with its "MOCK" or "RESEARCH_MODEL" label
<code>ai_inference_logs</code>	a technical log of each model run
<code>imaging_metadata</code>	technical fields of uploaded scans (never the patient's name)
<code>chatbot_conversations</code>	chats; deleted automatically 180 days after the last message

6.4 Looking at the data

- **PostgreSQL:** open **pgAdmin 4** > Servers > PostgreSQL 18 > Databases > `pca_mhealth` > Schemas > public > Tables. Right-click a table > View/Edit Data > All Rows.
- **MongoDB:** open **MongoDB Compass**, connect to `mongodb://localhost:27017`, open `pca_mhealth`.

Reading is always safe. **Do not add, change or delete rows by hand:** - A typed-in account cannot sign in, because passwords are stored as hashes and some fields are encrypted. - A hand-deleted row breaks links between records. - A hand-edited audit log fails its integrity check.

Use the admin website or the API instead. The full guide is [local-databases.md](#).

6.5 Backups

```
# PostgreSQL: one file
& "C:\Program Files\PostgreSQL\18\bin\pg_dump.exe" -h localhost -U pca -d pca_mhealth -Fc -f
D:\backups\pca_mhealth.dump

# Restore into an empty database
& "C:\Program Files\PostgreSQL\18\bin\pg_restore.exe" -h localhost -U pca -d pca_mhealth --no-owner -
-no-privileges D:\backups\pca_mhealth.dump
```

- **MongoDB:** copy it to another server or database name with the project's tool (below), or install the free *MongoDB Database Tools* for `mongodump` and `mongorestore`.
- **Files:** copy the folder `3-application-logic/backend/var/objects`.
- `.env`: keep a copy somewhere safe. Without its encryption keys, the encrypted fields in a backup cannot be read.

The copy tool walks through every collection and copies its rules, its indexes and its documents:

```

30   for (const info of collections) {
31     if (info.name.startsWith('system.')) continue;
32     const exists = await target
33       .listCollections({ name: info.name })
34       .hasNext();
35     if (!exists) {
36       const options: Document = ('options' in info && info.options) || {};
37       await target.createCollection(info.name, {
38         validator: options.validator as Document | undefined,
39         validationLevel:
40           options.validationLevel as CreateCollectionOptions['validationLevel'],
41         validationAction:
42           options.validationAction as CreateCollectionOptions['validationAction'],
43       });
44     }
45     const into = target.collection(info.name);
46     if ((await into.estimatedDocumentCount()) > 0) {
47       console.log(`${info.name}: already has documents, skipped`);
48       continue;
49     }
50     for (const index of await source.collection(info.name).indexes()) {
51       if (index.name === '_id_') continue;
52       const {
53         key,
54         v: _v,
55         ns: _ns,
56         ...rest
57       } = index as Record<string, unknown> & { key: Record<string, 1 | -1> };
58       await into.createIndex(key, rest);
59     }
60     const docs = await source.collection(info.name).find().toArray();
61     if (docs.length > 0) {
62       await into.insertMany(docs, { bypassDocumentValidation: true });
63     }
64     const copied = await into.countDocuments();
65     console.log(`${info.name}: ${docs.length} read, ${copied} in target`);
66   }
67 } finally {

```

6.6 How the databases were moved onto this PC

Until 29 September 2026 PostgreSQL and MongoDB ran inside Docker. They were moved to the servers installed on the PC in six steps: create the login and database, dump and restore PostgreSQL, create the MongoDB login, copy the collections, change two lines in `.env`, and switch the Docker copies off. Every table had the same number of rows afterwards, and the audit chain still verified. The exact commands, and how to switch back, are in [local-databases.md](#).

7. Docker

Docker runs a program in a sealed box called a **container**, with its data in a **volume** that survives restarts. This project uses Docker only for helper services, so nothing has to be installed for them.

`6-infrastructure/docker/docker-compose.yml` lists the containers:

```

1 # Development data stores for PCa mHealth. Not a production deployment.
2 # Usage (from the repository root): docker compose -f 6-infrastructure/docker/docker-compose.yml --env-file
  .env up -d
3 #
4 # PostgreSQL and MongoDB normally run as the services installed on the PC
5 # (PostgreSQL 18 on 5432, MongoDB on 27017; docs/local-databases.md), so here
6 # they are in the optional "docker-db" profile. To use Docker for them instead,
7 # add --profile docker-db and point DATABASE_URL / MONGO_URL at 5433 / 27018.
8 name: pca-mhealth
9
10 services:
11   postgres:
12     profiles: [docker-db]
13     image: postgres:16-alpine
14     environment:
15       POSTGRES_USER: ${POSTGRES_USER:-pca}
16       POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:?set POSTGRES_PASSWORD in .env}
17       POSTGRES_DB: ${POSTGRES_DB:-pca_mhealth}
18     ports: ["127.0.0.1:${POSTGRES_PORT:-5433}:5432"]
19     volumes: [pgdata:/var/lib/postgresql/data]
20     healthcheck:
21       test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER:-pca}"]
22       interval: 5s
23       retries: 10
24     restart: unless-stopped
25
26   mongo:
27     profiles: [docker-db]
28     image: mongo:7
29     environment:
30       MONGO_INITDB_ROOT_USERNAME: ${MONGO_USER:-pca}
31       MONGO_INITDB_ROOT_PASSWORD: ${MONGO_PASSWORD:?set MONGO_PASSWORD in .env}
32     ports: ["127.0.0.1:${MONGO_PORT:-27018}:27017"]
33     volumes: [mongodata:/data/db]
34     healthcheck:
35       test: ["CMD", "mongosh", "--quiet", "--eval", "db.adminCommand('ping')"]
36       interval: 5s
37       retries: 10
38     restart: unless-stopped
39
40   redis:
41     image: redis:7-alpine
42     command: ["redis-server", "--requirepass", "${REDIS_PASSWORD:?set REDIS_PASSWORD in .env}"]
43     ports: ["127.0.0.1:6379:6379"]
44     restart: unless-stopped
45
46   minio:
47     image: quay.io/minio/minio@sha256:14cea493d9a34af32f524e538b8346cf79f3321eff8e708c1e2960462bd8936e
48     command: ["server", "/data", "--console-address", ":9001"]
49     environment:
50       MINIO_ROOT_USER: ${MINIO_ROOT_USER:-pcaminio}

```

Service	What it is	Port	Used now?
redis	fast counters for rate limits	6379	yes
minio	S3-style file storage, with a web console on 9001	9000, 9001	by the database tests; by the app only when STORAGE_DRIVER=s3

Service	What it is	Port	Used now?
qdrant	vector search	6333	not yet (VECTOR_STORE=memory)
postgres , mongo	the old Docker databases, kept as a backup	5433, 27018	no: they are in the optional docker-db profile and stay off

Commands (Docker Desktop must be running; wait for "Engine running"):

```
$compose = "6-infrastructure\docker\docker-compose.yml"
```

```
docker compose -f $compose --env-file .env up -d      # start
docker compose -f $compose --env-file .env ps         # what is running
docker compose -f $compose --env-file .env logs redis # one service's messages
docker compose -f $compose --env-file .env restart minio
docker compose -f $compose --env-file .env stop       # stop, keep the data
```

Docker: what runs in containers

```
PS D:\Final Year Project\pca-mhealth> docker compose -f 6-infrastructure\docker\docker-compose.yml --env-file .env ps --format "table {{.Service}}\t{{.Status}}\t{{.Ports}}"
```

SERVICE	STATUS	PORTS
minio	Up 25 hours	127.0.0.1:9000-9001->9000-9001/tcp
qdrant	Up 25 hours	127.0.0.1:6333->6333/tcp
redis	Up 25 hours	127.0.0.1:6379->6379/tcp

- **stop keeps everything.** `down` removes the containers but keeps the volumes. `down -v` **deletes the volumes and their data: do not use it** unless you mean to.
- **To use the Docker databases again:** set the ports in `DATABASE_URL` and `MONGO_URL` to 5433 and 27018, then run `dev-up.ps1 -DockerDatabases`.
- **A password change** for Redis or MinIO needs the container recreated: `docker compose -f $compose --env-file .env up -d --force-recreate redis minio`.

8. The AI service

A separate Python program (FastAPI) in `4-ai-intelligence-layer/ai-services`. Only the backend may call it, with the shared `AI_SERVICE_TOKEN`.

Route	What it does
GET /v1/health	says it is up, that the models are development mocks , whether Claude writes chat answers, and how chat passages are found
GET /v1/models	the list of models and their versions
POST /v1/infer	runs an analysis (MRI, slide, risk score)
POST /v1/chat/answer	finds the reviewed passages for a question and returns the answer

The models are mocks. No trained model exists yet, so the service returns labelled development data ("DEVELOPMENT MOCK DATA — NOT A CLINICAL RESULT"), and the app shows that label. When real models are trained, only this service changes: see [ai-model-integration-guide.md](#).

Commands (in `4-ai-intelligence-layer\ai-services`):

Task	Command
Install (first time)	<code>py -3 -m venv .venv then .venv\Scripts\python -m pip install -e ".[dev]"</code>
Run	<code>.venv\Scripts\python -m uvicorn app.main:app --host 127.0.0.1 --port 8000</code>
Tests	<code>.venv\Scripts\python -m pytest -q</code>
Lint and types	<code>.venv\Scripts\python -m ruff check .</code> and <code>.venv\Scripts\python -m mypy</code>
Download the meaning-search models (once, about 880 MB)	<code>.venv\Scripts\python -m app.chat.embeddings download</code>
Measure the assistant's search on the question sets	<code>.venv\Scripts\python -m app.chat.evaluate</code>

9. The chatbot, end to end

9.1 The path of one question

1. The person types or speaks a question in the app. Speech is turned into text by the phone.
2. The app sends the text to the backend: `POST /api/v1/chat/conversations/{id}/messages`.
3. The backend checks the permission (`chatbot:use`) and the limit (30 questions an hour per account).
4. **Safety rules** look at the question. An emergency, self-harm, a medicine dose, or a patient asking what their own results mean gets a fixed, careful reply. Nothing is looked up.
5. **Small talk** ("hello", "how are you", "tell me a joke") gets a friendly fixed reply.
6. Otherwise the backend asks the AI service, which **searches the reviewed knowledge base**: it finds the passages that share the question's key words, and puts them in order of meaning.
7. If Claude is switched on, Claude writes a short answer **from those passages only**. If not, or if anything fails, the passages are quoted word for word.
8. The backend checks the answer: it must have a source and must not contain a dose.
9. Question and answer are stored in MongoDB. The audit log records that a question was asked, never its words.
10. The app shows the answer with its sources, the content's review status and a disclaimer.

Steps 4 to 8 are this one function:

```

270  /** Safety check, then the quoted answer, then the output check. */
271  private async answer(
272    text: string,
273    audience: ChatAudience,
274    language: string,
275    earlier: StoredMessage[],
276  ): Promise<{ answer: StoredMessage; reason: string }> {
277    const fixed = (safety: ChatSafety, fixedText: string, reason: string) => ({
278      answer: {
279        id: randomUUID(),
280        role: 'assistant' as const,
281        text: fixedText,
282        at: new Date(),
283        sources: [],
284        safety,
285        mode: 'FIXED' as const,
286      },
287      reason,
288    });
289    const decision = checkQuestion(text, audience);
290    if (decision.safety !== 'OK') {
291      return fixed(decision.safety, decision.text!, decision.reason);
292    }
293    // The number of earlier questions varies repeated replies ("Hi again!",
294    // another joke).
295    const friendly = smallTalk(
296      text,
297      audience,
298      earlier.filter((m) => m.role === 'user').length,
299    );
300    if (friendly) return fixed('OK', friendly, 'small-talk');
301    const noSource =
302      audience === 'patient'
303        ? FIXED_TEXT.noSourcePatient
304        : FIXED_TEXT.noSourceClinician;
305    let result;
306    try {
307      result = await this.ai.chatAnswer({
308        question: text,
309        audience,
310        language,
311        // The last three questions and answers, for follow-ups.
312        history: earlier.slice(-6).map((m) => ({ role: m.role, text: m.text })),
313      });
314    } catch (err) {
315      if (err instanceof AiServiceError && err.status === 409) {
316        throw new ConflictException({
317          code: 'LANGUAGE_NOT_AVAILABLE',
318          message: 'There is no verified content in this language yet.',
319        });
320      }
321      throw new ServiceUnavailableException({
322        code: 'CHAT_UNAVAILABLE',
323        message: 'The assistant is not available right now. Please try later.',
324      });
325    }
326    if (!result.matched) return fixed('NO_SOURCE', noSource, 'no-match');
327    const check = checkAnswer(result.text, result.sources.length);

```

9.2 The safety rules

They are plain pattern checks in `src/services/chatbot/chat-safety.ts`, so they behave the same every time and can be tested:

```
52 export function checkQuestion(  
53   text: string,  
54   audience: ChatAudience,  
55 ): SafetyDecision {  
56   if (SELF_HARM.test(text)) {  
57     return {  
58       safety: 'URGENT_CARE',  
59       reason: 'self-harm',  
60       text: FIXED_TEXT.urgentSelfHarm,  
61     };  
62   }  
63   if (EMERGENCY.test(text)) {  
64     return {  
65       safety: 'URGENT_CARE',  
66       reason: 'emergency',  
67       text: FIXED_TEXT.urgentMedical,  
68     };  
69   }  
70   if (MEDICINES.test(text)) {  
71     return {  
72       safety: 'DECLINED',  
73       reason: 'medicines',  
74       text: FIXED_TEXT.medicines,  
75     };  
76   }  
77   // Clinicians may ask what a value means in general; patients are never  
78   // told what their own results mean (their clinician explains them).  
79   if (audience === 'patient' && OWN_RESULTS.test(text)) {  
80     return {  
81       safety: 'DECLINED',  
82       reason: 'own-results',  
83       text: FIXED_TEXT.ownResults,  
84     };  
85   }  
86   return { safety: 'OK', reason: 'ok' };  
87 }
```

9.3 Small talk

There are 18 kinds of casual message, each with a reply for patients and one for clinicians:

```

3-application-logic/backend/src/services/chatbot/chat-safety.ts
lines 141-170

141     name: 'how-are-you',
142     pattern: whole(
143       String.raw`how (?are|r) (?you|u|things)(?: (?doing|today|keeping|going|this
morning|tonight))*|how(?:'s| is) it going|how(?:'s| is) your day|how do you do|are you (?ok(?:ay)?
|well|fine|alright|good)|what's up|wh?assup|sup`,
144     ),
145     replies: {
146       patient: [
147         'I am doing well, thank you for asking! I am here to help with questions about ' +
148           HELP_TOPICS_PATIENT +
149         '. How can I help you today?',
150       ],
151       clinician: ['Doing well, thank you. What would you like to look up?'],
152     },
153   },
154   {
155     name: 'greeting',
156     pattern: whole(
157       String.raw`hi+|hello+|hey+|hallo|hiya|howdy|yo|greetings|good
(?:morning|afternoon|evening|day)|morning|evening|hi there|hello there|hello hello`,
158     ),
159     replies: {
160       patient: [
161         'Hello! I can answer questions about ' +
162           HELP_TOPICS_PATIENT +
163         '. What would you like to know?',
164         'Hi again! What else would you like to know? You can ask about ' +
165           HELP_TOPICS_PATIENT +
166         '.',
167       ],
168       clinician: [
169         'Hello. I can look up reference information: PI-RADS categories, ISUP grade groups, PSA density and
free PSA, DRE findings, and how to read the AI report. What do you need?',
170       ],

```

To add a phrase, add it to the `pattern` of the right intent, or add a new block to the `INTENTS` list in `chat-safety.ts`. Then add a line to the table in `chat-safety.spec.ts` and run:

```

cd 3-application-logic\backend
node node_modules\jest\bin\jest.js src/services/chatbot

```

Only a message that is *nothing but* small talk counts: "hello, is my PSA bad?" still gets the safety reply.

9.4 The knowledge base

The assistant can only say what is in two files in `4-ai-intelligence-layer/knowledge-base/en/` : - `patient-learn.json` : the app's Learn articles, word for word; - `clinician-reference.json` : reference cards (PI-RADS, grade groups, PSA density, DRE findings, reading the AI report).

```

1 {
2   "audience": "patient",
3   "language": "en",
4   "reviewStatus": "Draft for review by a qualified clinician before use with patients (research prototype content).",
5   "articles": [
6     {
7       "id": "prostate",
8       "title": "What is the prostate?",
9       "summary": "A small gland that helps make semen, and why it can cause problems as men get older.",
10      "minutes": 2,
11      "sections": [
12        {
13          "heading": "Where it is and what it does",
14          "body": "The prostate is a small gland, about the size of a walnut, found only in men. It sits below the bladder and surrounds the tube that carries urine out of the body (the urethra). It helps to make the fluid that carries sperm (semen).",
15        },
16        {
17          "heading": "Changes with age",
18          "body": "The prostate often gets bigger as men get older. This is common and is usually not cancer. A bigger prostate can press on the urethra and cause changes in passing urine, such as needing to go more often, especially at night, or a weaker flow.",
19        },
20        {
21          "heading": "Prostate cancer",
22          "body": "Prostate cancer happens when cells in the prostate grow out of control. Some prostate cancers grow very slowly and may never cause harm; others can grow and spread. Finding out which kind a man has needs tests and a clinician's assessment.",
23        }
24      ],
25      "sources": [
26        {

```

Every section of every article is one **passage** the assistant can use. Passages are found in two steps ([ADR-013](#)):

1. **Keywords (BM25) decide whether the question is covered.** A passage must share enough of the question's words (a score of 2.0 or more). This needs no model, and every match can be explained by the words it shares with the question.
2. **Meaning (MedCPT) puts those passages in order.** A biomedical model from the US National Library of Medicine turns the question and each passage into 768 numbers, and the passage closest in meaning comes first. If even the closest one is far from the question (a score under 52.0), the answer is "no reviewed information". - "Are African men more likely to get it?" now gets the passage on who is at higher risk. The keywords alone preferred the one about the gland. - "What is the treatment for malaria?" is refused. It only shares the word "treatment" with a screening passage.

```

65 class Retriever:
66     """BM25 over the passages one audience may see."""
67
68     K1 = 1.5
69     B = 0.75
70
71     def __init__(self, kb: KnowledgeBase, audience: Audience, language: str) -> None:
72         # Clinicians may also be given the plain patient information.
73         allowed = {"patient"} if audience == "patient" else {"patient", "clinician"}
74         self.passages = [p for p in kb.passages if p.audience in allowed and p.language == language]
75         # The title and heading count twice: they say what the passage is about.
76         self._docs = [
77             terms(f"{p.title} {p.heading} {p.title} {p.heading} {p.body}") for p in self.passages
78         ]
79         self._freq = [Counter(d) for d in self._docs]
80         n = len(self._docs)
81         self._avg_len = sum(len(d) for d in self._docs) / n if n else 0.0
82         df: Counter[str] = Counter()
83         for d in self._docs:
84             df.update(set(d))
85         self.idf = {t: math.log(1 + (n - c + 0.5) / (c + 0.5)) for t, c in df.items()}
86
87     def search(self, question: str, top_k: int = 5) -> list[Hit]:
88         query = query_terms(question)
89         hits: list[Hit] = []
90         for passage, doc, freq in zip(self.passages, self._docs, self._freq, strict=True):
91             score = 0.0
92             for t in query:
93                 f = freq.get(t, 0)
94                 if f:
95                     norm = self.K1 * (1 - self.B + self.B * len(doc) / self._avg_len)
96                     score += self.idf[t] * f * (self.K1 + 1) / (f + norm)
97             if score > 0:
98                 hits.append(Hit(passage, score))
99         hits.sort(key=lambda h: h.score, reverse=True)
100         return hits[:top_k]

```

```

131 def _rank(
132     retriever: Retriever, question: str, text: str, meaning: MeaningScorer | None
133 ) -> tuple[list[Hit], list[Hit]] | None:
134     """The passages in order of relevance, and the ones to quote; None when the
135     knowledge base does not cover the question. `text` is the question, or a
136     short follow-up read together with the previous question."""
137     if meaning is None:
138         hits = retriever.search(text)
139         if not hits or hits[0].score < MIN_SCORE:
140             return None
141         # Whole passages are quoted: each knowledge base section is written to
142         # stand on its own, so a quote never loses the sentence it depends on.
143         quoted = [hits[0]]
144         if (
145             len(hits) > 1
146             and hits[1].score >= SECOND_PASSAGE_RATIO * hits[0].score
147             and len(hits[0].passage.body) + len(hits[1].passage.body) <= MAX_CHARS
148         ):
149             quoted.append(hits[1])
150         return hits, quoted
151
152     # Meaning search: the keywords decide whether the question is covered;
153     # among the passages that match them, the closest in meaning comes first.
154     hits = retriever.search(text, top_k=len(retriever.passages))
155     candidates = [h for h in hits if h.score >= MIN_SCORE]
156     if not candidates:
157         return None
158     closeness = meaning.scores(text, [h.passage for h in candidates])
159     ordered = sorted(zip(candidates, closeness, strict=True), key=lambda pair: -pair[1])
160     (best, best_closeness) = ordered[0]
161     if best_closeness < MEANING_FLOOR:
162         return None
163     # A short question read with the previous one must itself be close to the
164     # passage: "Does it hurt?" after the DRE is, "What is the treatment for
165     # malaria?" after the DRE is not (it is a new topic, and not covered).
166     if text != question and meaning.scores(question, [best.passage])[0] < MEANING_FLOOR:
167         return None
168     quoted = [best]
169     if (
170         len(ordered) > 1
171         and ordered[1][1] >= best_closeness - MEANING_SECOND_GAP
172         and len(best.passage.body) + len(ordered[1][0].passage.body) <= MAX_CHARS
173     ):
174         quoted.append(ordered[1][0])
175     ranked = [h for h, _ in ordered] + [h for h in hits if h.score < MIN_SCORE]
176     return ranked, quoted

```

The model runs inside the AI service, without the internet. It is a 12-layer BERT encoder written with numpy, so it needs no PyTorch:

```

171     def _encode(self, ids: Any, mask: Any, types: Any) -> Any:
172         np = self._np
173         batch, length = ids.shape
174         h = (
175             self.w["embeddings.word_embeddings.weight"][ids]
176             + self.w["embeddings.position_embeddings.weight"][:length]
177             + self.w["embeddings.token_type_embeddings.weight"][types]
178         )
179         h = self._norm(h, "embeddings.LayerNorm")
180         size = h.shape[-1] // self.heads
181         # Padding gets no attention.
182         blocked = ((1.0 - mask[:, None, None, :]) * np.finfo(np.float32).min).astype(np.float32)
183
184     def heads(x: Any) -> Any:
185         return x.reshape(batch, length, self.heads, size).transpose(0, 2, 1, 3)
186
187     for i in range(self.layers):
188         p = f"encoder.layer.{i}"
189         q = heads(self._linear(h, f"{p}.attention.self.query"))
190         k = heads(self._linear(h, f"{p}.attention.self.key"))
191         v = heads(self._linear(h, f"{p}.attention.self.value"))
192         scores = q @ k.transpose(0, 1, 3, 2) / math.sqrt(size) + blocked
193         scores = np.exp(scores - scores.max(-1, keepdims=True))
194         probs = scores / scores.sum(-1, keepdims=True)
195         context = (probs @ v).transpose(0, 2, 1, 3).reshape(batch, length, -1)
196         h = self._norm(
197             self._linear(context, f"{p}.attention.output.dense") + h,
198             f"{p}.attention.output.LayerNorm",
199         )
200         inner = self._linear(h, f"{p}.intermediate.dense")
201         inner = 0.5 * inner * (1.0 + _erf(inner / math.sqrt(2.0)))
202         h = self._norm(self._linear(inner, f"{p}.output.dense") + h, f"{p}.output.LayerNorm")
203     return h

```

`answer.py` then decides: no passage left means "no reviewed information"; with Claude, Claude writes from the three best passages; otherwise the best passages are quoted.

```

179 def answer(
180     kb: KnowledgeBase,
181     request: ChatAnswerRequest,
182     writer: ClaudeWriter | None = None,
183     meaning: MeaningScorer | None = None,
184 ) -> ChatAnswerResult:
185     if request.language not in kb.languages():
186         raise LanguageNotAvailableError(request.language)
187     audience: Audience = request.audience
188     retriever = Retriever(kb, audience, request.language)
189     info = KnowledgeBaseInfo(version=kb.version, reviewStatus=kb.review_status)
190     unmatched = ChatAnswerResult(
191         matched=False, text=None, passages=[], sources=[], knowledgeBase=info
192     )
193     text = _search_text(request)
194     try:
195         ranking = _rank(retriever, request.question, text, meaning)
196     except MeaningBusyError:
197         # Every slot of the meaning search is taken (heavy load): keywords
198         # only, as before ADR-013, so the answer still comes within 2 s.
199         ranking = _rank(retriever, request.question, text, None)
200     if ranking is None:
201         return unmatched
202     hits, quoted = ranking
203
204     if writer is not None:
205         given = hits[:PASSAGES_FOR_CLAUDE]
206         written = writer.write(
207             request.question,
208             audience,
209             [h.passage for h in given],
210             [Turn(role=t.role, text=t.text) for t in request.history],
211         )
212         if written is not None and not written.covered:
213             return unmatched.model_copy(update={"mode": "GENERATED", "model": written.model})
214         if written is not None and written.cited:
215             used = [given[i] for i in written.cited]
216             return ChatAnswerResult(
217                 mode="GENERATED",
218                 matched=True,
219                 text=written.text,
220                 passages=[_passage_out(h) for h in used],
221                 sources=_sources([h.passage for h in used]),
222                 knowledgeBase=_info(kb, [h.passage for h in used]),
223                 model=written.model,
224             )
225         # No usable answer from Claude: quote the passages instead.
226
227     return ChatAnswerResult(
228         matched=True,
229         text=" ".join(h.passage.body for h in quoted),
230         passages=[_passage_out(h) for h in quoted],
231         sources=_sources([h.passage for h in quoted]),
232         knowledgeBase=_info(kb, [h.passage for h in quoted]),
233     )

```

Setting up the meaning search (once). In `4-ai-intelligence-layer\ai-services`, run `.venv\Scripts\python -m app.chat.embeddings download`. - It fetches the two MedCPT encoders (about 880 MB) into `models\`. Git ignores that folder. - It refuses any file whose checksum differs from the pinned one. - Then restart the AI service. A few seconds later, the health check says `chat_retrieval: keywords+meaning (MedCPT)`. - Without the models, the assistant answers with keywords only, as it did before. `dev-up.ps1` reminds you to download them.

Under heavy load the meaning search handles 6 questions at a time on this laptop (`CHAT_MEANING_SLOTS`). A question that waits more than half a second for its turn (`CHAT_MEANING_WAIT_MS`) is answered with keywords only, so answers stay within 2 seconds. The health check counts those questions (`chat_meaning_busy`). With 100 clinicians asking at once, none needed it ([performance.md](#)).

To change what the assistant knows:

1. Edit the JSON file. Keep each section short and complete in itself, and give every article a source.
2. If you changed `patient-learn.json`, copy it over `1-presentation-layer/mobile-app/assets/education/en/articles.json`. A test fails if the two differ.
3. Restart the AI service. It reads the files when it starts, and works out the new passages' meaning vectors in the background (about 12 seconds).
4. Run its tests: `.venv\Scripts\python -m pytest -q`.
5. Check the search on the question sets: `.venv\Scripts\python -m app.chat.evaluate`. It shows which questions changed their answer, and the meaning scores next to the 52.0 floor. Add new kinds of questions to `app/chat/eval_sets.py`.

The content says *"Draft for review by a qualified clinician"* until a clinician signs it off. Record a sign-off by changing `reviewStatus` in the file and noting the reviewer and date in the development log.

9.5 Claude (optional, costs money)

Claude is **off** on this PC: the health check says `chat_writer: quotes`. To switch it on:

1. Create an API key at console.anthropic.com.
2. Open `.env` and put the key after `ANTHROPIC_API_KEY=`. Never put it in a chat, an email or the code.
3. Restart the AI service. The health check then says `chat_writer: claude:claude-haiku-4-5-20251001`.

What Claude is told:

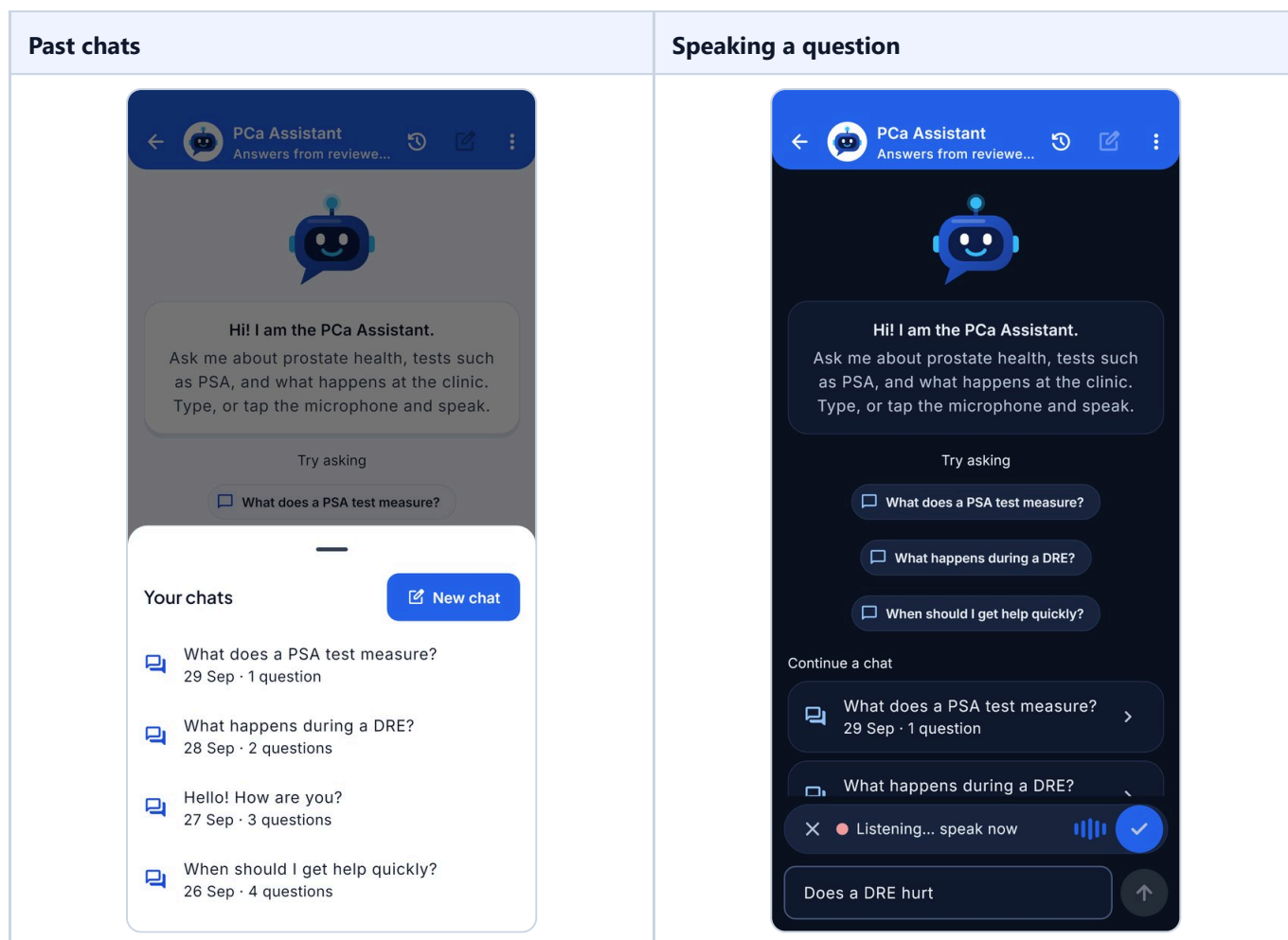
```
4-ai-intelligence-layer/ai-services/app/chat/generate.py lines 50-64
50 SYSTEM: dict[Audience, str] = {
51     "patient": (
52         "You are the PCa mHealth assistant, answering questions from patients in Zambia about "
53         "the prostate, PSA tests, the digital rectal exam, screening and when to get help. "
54         "Write warmly and simply: short sentences, everyday words, a reading age of about 12, "
55         "at most about 120 words. When it helps, suggest a question they could ask their "
56         "clinician." + _COMMON_RULES
57     ),
58     "clinician": (
59         "You are the PCa mHealth reference assistant for clinicians in Zambia. Answer concisely "
60         "and precisely, at most about 150 words, and keep all definitions and numbers exactly as "
61         "in the passages. The answer supports but does not replace clinical judgement or the "
62         "local protocol." + _COMMON_RULES
63     ),
64 }
```

- Claude only receives the three best passages, the last three questions and answers, and the question. Email addresses, NRC numbers and phone numbers are removed first.
- It must answer through a fixed form (`covered`, `answer`, `citations`). Only passages it cites become the sources.

- Limits: 8 seconds, no retries, at most `CHAT_LLM_DAILY_LIMIT` (2,000) answers a day. On any problem the passages are quoted instead, so the chat works without Claude.
- Answers written by Claude are labelled *"Written by AI (Claude) from the sources below"*.
- **Before real patients:** questions would be sent to Anthropic in the United States. That needs a data-protection review first (ADR-010).

9.6 What the person sees

Empty chat	An answer with its source	Casual chat



These screens are drawn by the app's own code with synthetic data. The answer is the AI service's real answer to that question (quoted, because Claude is off), and the casual replies are the backend's real replies. In the recording picture a stand-in plays the part of the phone's speech service.

9.7 Testing the chatbot

What	Command	Where
Safety rules and small talk (89 tests)	<code>node node_modules\jest\bin\jest.js src/services/chatbot</code>	3-application-logic\backend
The whole chat through the API and MongoDB (11 tests)	<code>node node_modules\jest\bin\jest.js --config ./test/jest-db.json --runInBand test/db/chatbot.int-spec.ts</code>	3-application-logic\backend
Search quality (keywords and meaning), and Claude with a stand-in	<code>.venv\Scripts\python -m pytest -q</code>	4-ai-intelligence-layer\ai-services
The search measured on the question sets	<code>.venv\Scripts\python -m app.chat.evaluate</code>	4-ai-intelligence-layer\ai-services
The chat screens	<code>flutter test test/features/chat</code>	1-presentation-layer\mobile-app

10. The phone app

10.1 How it is organised

1-presentation-layer/mobile-app/lib :

Folder	What is inside
main.dart	the first code that runs
app/	the routes (which screen is at which address, and who may open it) and the theme (colours, type, shapes)
core/	shared machinery: <code>network</code> (talking to the API), <code>db</code> (the encrypted database on the phone), <code>sync</code> (sending offline work later), <code>storage</code> (the session tokens), <code>uploads</code>
features/	one folder per feature: <code>auth</code> , <code>home</code> , <code>patients</code> , <code>clinical_server</code> , <code>patient</code> , <code>chat</code> , <code>sync</code> , <code>settings</code>
shared/	widgets used everywhere (the header, tiles, the assistant's face) and <code>audio</code> (read-aloud)

`main.dart` opens the phone's encrypted database and the saved preferences, then starts the app:

1-presentation-layer/mobile-app/lib/main.dartlines 12–26

```
12 Future<void> main() async {
13   WidgetsFlutterBinding.ensureInitialized();
14   _registerFontLicences();
15   final database = await openAppDatabase();
16   final preferences = await SharedPreferences.getInstance();
17   runApp(
18     ProviderScope(
19       overrides: [
20         appDatabaseProvider.overrideWithValue(database),
21         preferencesProvider.overrideWithValue(preferences),
22       ],
23       child: const PcaApp(),
24     ),
25   );
26 }
```

The only setting built into the app is the server's address. No secret is ever compiled in:

```

1  /// Build-time configuration. No secrets are ever compiled into the app:
2  /// only the API address, set with '--dart-define=API_BASE_URL=...'.
3  abstract final class AppEnv {
4    /// Defaults to the host machine as seen from the Android emulator.
5    static const apiBaseUrl = String.fromEnvironment(
6      'API_BASE_URL',
7      defaultValue: 'http://10.0.2.2:3000',
8    );
9
10   /// Where administrators manage the system (a separate web app).
11   static const adminPortalUrl = String.fromEnvironment(
12     'ADMIN_PORTAL_URL',
13     defaultValue: 'http://localhost:5173',
14   );
15
16   static String get apiRoot =>
17     '${apiBaseUrl.replaceAll(RegExp(r'/' + '$'), '')}/api/v1';
18 }

```

10.2 Running the app

On the emulator (the backend must be running):

```

flutter emulators --launch Galaxy_S9_Plus_API_29
cd 1-presentation-layer\mobile-app
flutter pub get      # only after pubspec.yaml changed
flutter run -d emulator-5554 --dart-define=API_BASE_URL=http://10.0.2.2:3000

```

10.0.2.2 is how the emulator reaches this PC. While `flutter run` is open, press **r** to reload after a code change, **R** to restart the app, **q** to quit. In Android Studio, the run configuration **App - S9+ emulator** does the same.

On a real Android phone by USB:

1. On the phone: Settings > Developer options > USB debugging on. Plug it in and tap **Allow**.
2. `powershell -ExecutionPolicy Bypass -File 6-infrastructure\scripts\phone-usb.ps1` (it lets the phone reach the PC's port 3000).
3. `flutter run --dart-define=API_BASE_URL=http://localhost:3000`, or the Android Studio configuration **App - USB phone**.

To make an installable file (APK):

```
flutter build apk --release --dart-define=API_BASE_URL=https://your-server.example
```

The file appears in `build\app\outputs\flutter-apk\app-release.apk`. A release build for the public also needs a signing key and an HTTPS server address.

10.3 How the app is put together

- **State** is managed with Riverpod: a *controller* holds the state of one screen and the screen redraws when it changes. The chat controller is a typical example:

```
1-presentation-layer/mobile-app/lib/features/chat/application/chat_controller.dart lines 82–117

82 class ChatController extends Notifier<ChatState> {
83   @override
84   ChatState build() => const ChatState();
85
86   Future<void> ask(String text) async {
87     final question = text.trim();
88     if (question.isEmpty || state.sending || state.loading) return;
89     state = state.copyWith(sending: true, clearError: true);
90     final api = ref.read(chatApiProvider);
91     try {
92       final id = state.conversationId ?? await api.start();
93       final (asked, answer) = await api.ask(id, question);
94       state = state.copyWith(
95         conversationId: id,
96         messages: [...state.messages, asked, answer],
97         sending: false,
98       );
99       ref.invalidate(chatHistoryProvider);
100   } catch (e) {
101     state = state.copyWith(sending: false, error: chatErrorText(e));
102   }
103 }
104
105 /// Opens an earlier conversation so it can be read and continued.
106 Future<void> open(String conversationId) async {
107   if (state.sending || conversationId == state.conversationId) return;
108   state = ChatState(conversationId: conversationId, loading: true);
109   try {
110     final messages = await ref.read(chatApiProvider).messages(conversationId);
111     if (state.conversationId != conversationId) return;
112     state = ChatState(conversationId: conversationId, messages: messages);
113   } catch (e) {
114     if (state.conversationId != conversationId) return;
115     state = ChatState(error: chatErrorText(e));
116   }
117 }
```

- **Talking to the server:** `core/network/api_client.dart` adds the session token to every request, renews it when it has expired, and turns server errors into plain messages.
- **Offline:** clinicians' work is saved first in an encrypted database on the phone (Drift with SQLCipher) and sent when the connection returns (`core/sync`). The server re-checks every change with the same rules as online.
- **Session tokens** are kept in the phone's secure storage, never in plain files.
- **Permissions:** the app only asks for the internet and, when the microphone is first tapped, the microphone.

10.4 Voice and reading aloud

Both use the phone's own speech services, behind small interfaces, so tests can replace them with stand-ins.

Voice (`features/chat/application/voice_input.dart`): the phone turns speech into text, and the app only receives the words.

```
1-presentation-layer/mobile-app/lib/features/chat/application/voice_input.dart lines 74–102

74 Future<void> listen({
75   required void Function(String words) onWords,
76   required void Function(double level) onLevel,
77   required void Function() onDone,
78 }) async {
79   _onDone = onDone;
80   // The answers are in English: use the phone's English locale, or
81   // British English when the phone is set to another language.
82   final system = await _speech.systemLocale();
83   final locale = system != null && system.localeId.startsWith('en')
84     ? system.localeId
85     : 'en_GB';
86   await _speech.listen(
87     onResult: (result) {
88       onWords(result.recognizedWords);
89       if (result.finalResult) _finish();
90     },
91     // Android reports roughly -2 to 10 dB.
92     onSoundLevelChange: (level) => onLevel(((level + 2) / 12).clamp(0, 1)),
93     listenOptions: stt.SpeechListenOptions(
94       listenFor: const Duration(seconds: 60),
95       pauseFor: const Duration(seconds: 3),
96       localeId: locale,
97       partialResults: true,
98       cancelOnError: true,
99       listenMode: stt.ListenMode.dictation,
100   ),
101 );
102 }
```

Read-aloud (`shared/audio/read_aloud.dart`): an article is read part by part, so the screen can highlight the part being read.

```
110 class ReadAloudController extends Notifier<ReadAloudState> {
111   int _run = 0;
112   List<String> _parts = const [];
113
114   ReadAloud get _voice => ref.read(readAloudProvider);
115
116   @override
117   ReadAloudState build() => const ReadAloudState();
118
119   Future<void> play(String id, List<String> parts, {int from = 0}) async {
120     final run = ++_run;
121     await _voice.stop();
122     _parts = parts;
123     if (!ref.mounted) return;
124     await _voice.setRate(state.slow ? slowReadingRate : normalReadingRate);
125     for (var i = from; i < parts.length; i++) {
126       // Stopped, or another text started, or the app is closing.
127       if (!ref.mounted || run != _run) return;
128       state = ReadAloudState(
129         id: id,
130         part: i,
131         total: parts.length,
132         slow: state.slow,
133       );
134       await _voice.speak(parts[i]);
135     }
136     if (ref.mounted && run == _run) state = ReadAloudState(slow: state.slow);
137   }
138
139   Future<void> pause() async {
```

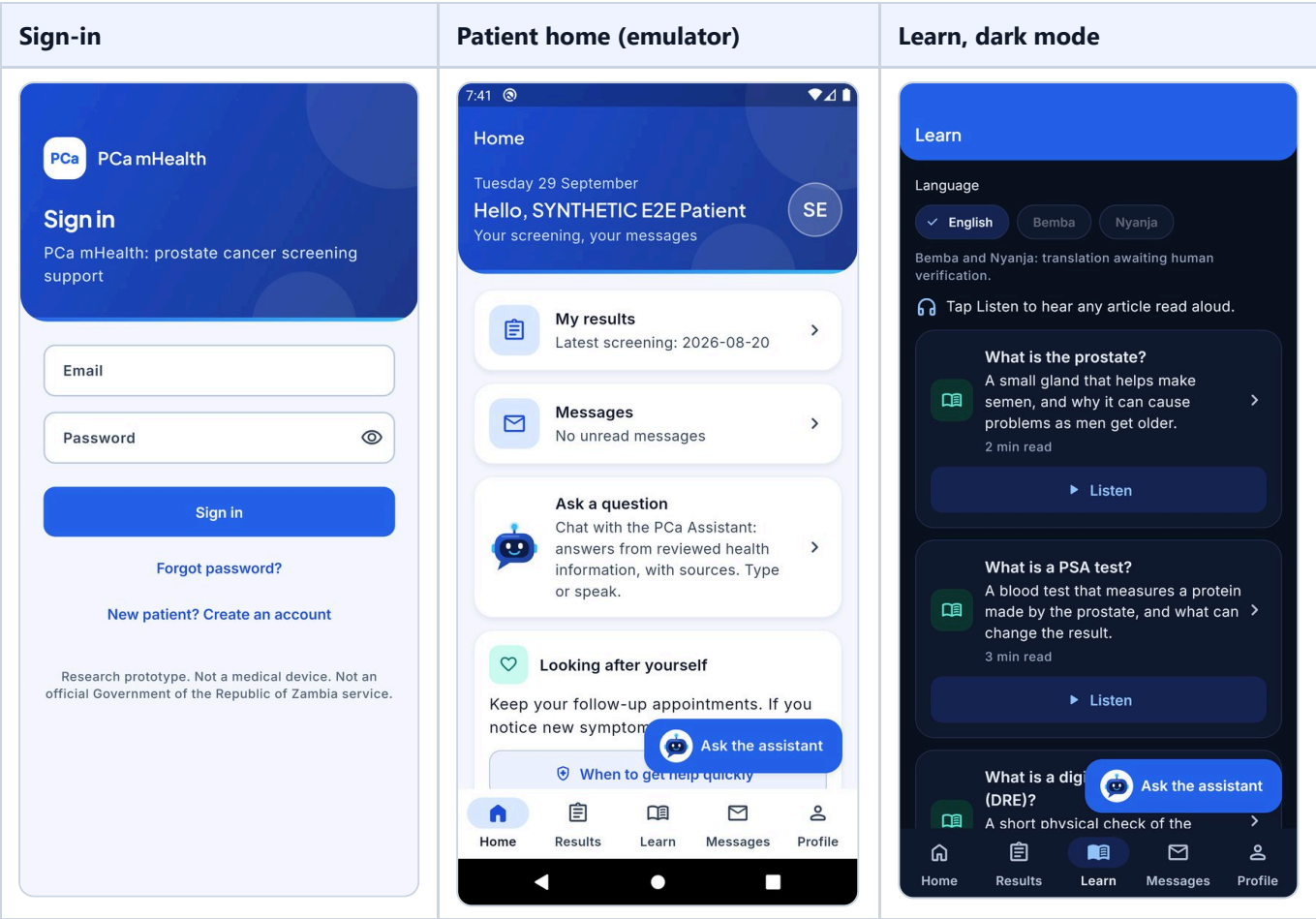
The assistant's face is not a picture file. It is drawn by code, so it is sharp at every size:

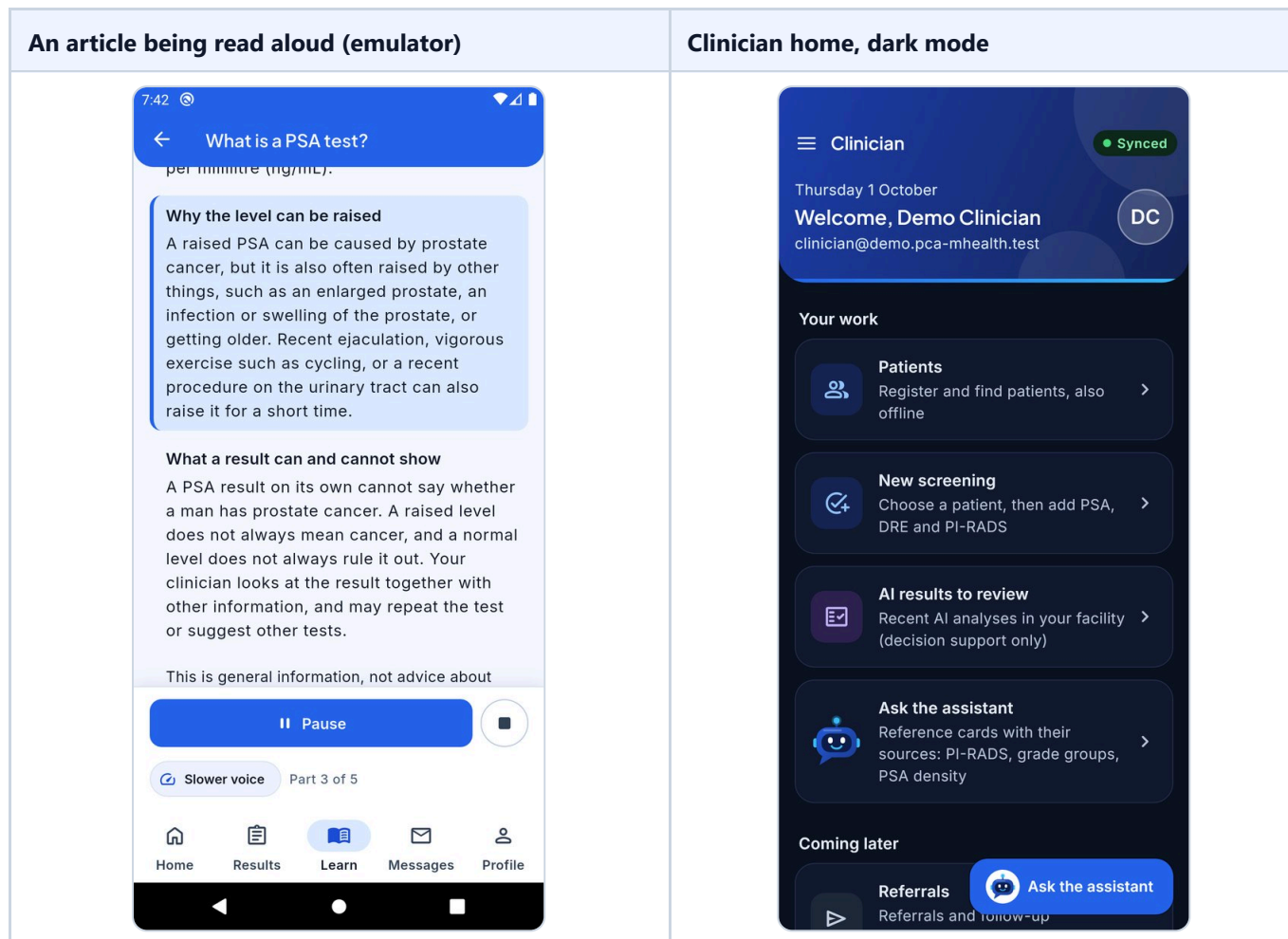
```

110 void paint(Canvas canvas, Size size) {
111   final u = size.shortestSide / 100;
112   Offset o(double x, double y) => Offset(x * u, y * u);
113   Rect box(double l, double t, double r, double b) =>
114     Rect.fromLTRB(l * u, t * u, r * u, b * u);
115
116   // Antenna with a glowing awareness-blue tip.
117   canvas.drawLine(
118     o(50, 25),
119     o(50, 13),
120     Paint()
121       ..color = _head.last
122       ..strokeWidth = 4.5 * u
123       ..strokeCap = StrokeCap.round,
124   );
125   canvas.drawCircle(
126     o(50, 10),
127     9.5 * u,
128     Paint()..color = _accent.withValues(alpha: 0.3),
129   );
130   canvas.drawCircle(o(50, 10), 6 * u, Paint()..color = _accent);
131
132   // Ears.
133   final ear = Paint()..color = _accent;
134   for (final (l, r) in [(5.0, 13.0), (87.0, 95.0)]) {
135     canvas.drawRRect(
136       RRect.fromRectAndRadius(box(l, 44, r, 62), Radius.circular(4 * u)),
137       ear,
138     );
139   }
140
141   // Head and speech-bubble tail, one shape with a blue gradient.
142   final head = Path()
143     ..addRRect(
144       RRect.fromRectAndRadius(box(11, 23, 89, 81), Radius.circular(22 * u)),
145     );
146   final tail = Path()
147     ..moveTo(24 * u, 74 * u)
148     ..lineTo(17 * u, 96 * u)
149     ..lineTo(44 * u, 79 * u)
150     ..close();
151   canvas.drawPath(
152     Path.combine(PathOperation.union, head, tail),
153     Paint()

```

10.5 What it looks like





10.6 App commands

Run these in `1-presentation-layer\mobile-app` :

Task	Command
Get packages	<code>flutter pub get</code>
Check the code	<code>flutter analyze</code>
Format	<code>dart format lib test</code>
All tests	<code>flutter test</code>
One test file	<code>flutter test test/features/chat/chat_test.dart</code>
Build a debug APK	<code>flutter build apk --debug</code>
List devices	<code>flutter devices</code>

11. The admin website

`1-presentation-layer/admin-panel-web` , a React application. Only administrators can sign in.

Page	What it is for
Dashboard	what is happening in the phone app: sign-ins, records, sync health (counts only, no patient names)
Users	every account: roles, facility, switch off, unlock, reset password, delete
Roles & permissions	what each role may do
Patient accounts	link a patient's sign-in to their clinic record
FHIR export	export records for SmartCare Pro
Audit log	who did what, and Check integrity

OVERVIEW

Dashboard

PEOPLE

Users

Roles & permissions

Patient accounts

RECORDS & DATA

FHIR export

Audit log

Dashboard

Update

What is happening in the phone app and on this website. Counts only; no patient names or results.

24 hours

7 days

30 days

90 days



Active in the phone app

6

people · 4 phones synced · 9 active in all

Sign-ins

31

2 failed

Patients registered

12

Screening records

18

AI analyses

4

4 finished

Scans and slides

5

Consents recorded

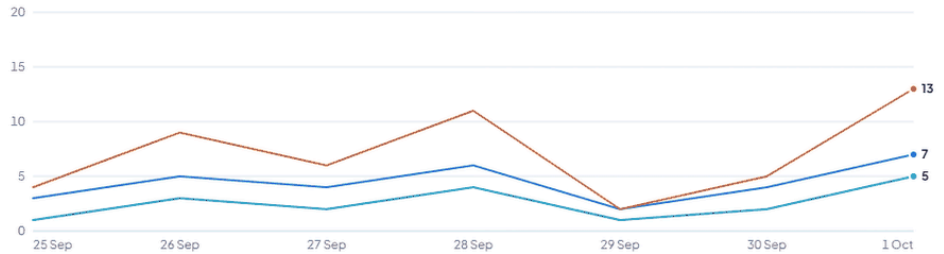
7

1 withdrawn

Activity per day

Days in Zambian time.

Sign-ins Changes synced from phones Screening records

[Show as table](#)

Where activity comes from

Recorded actions, by app.



Offline sync health

Changes the phones sent after working offline.

✓ Saved	50
⚠ Conflicts (kept for a person to choose)	2
✗ Refused as invalid	1

Sign-ins by role

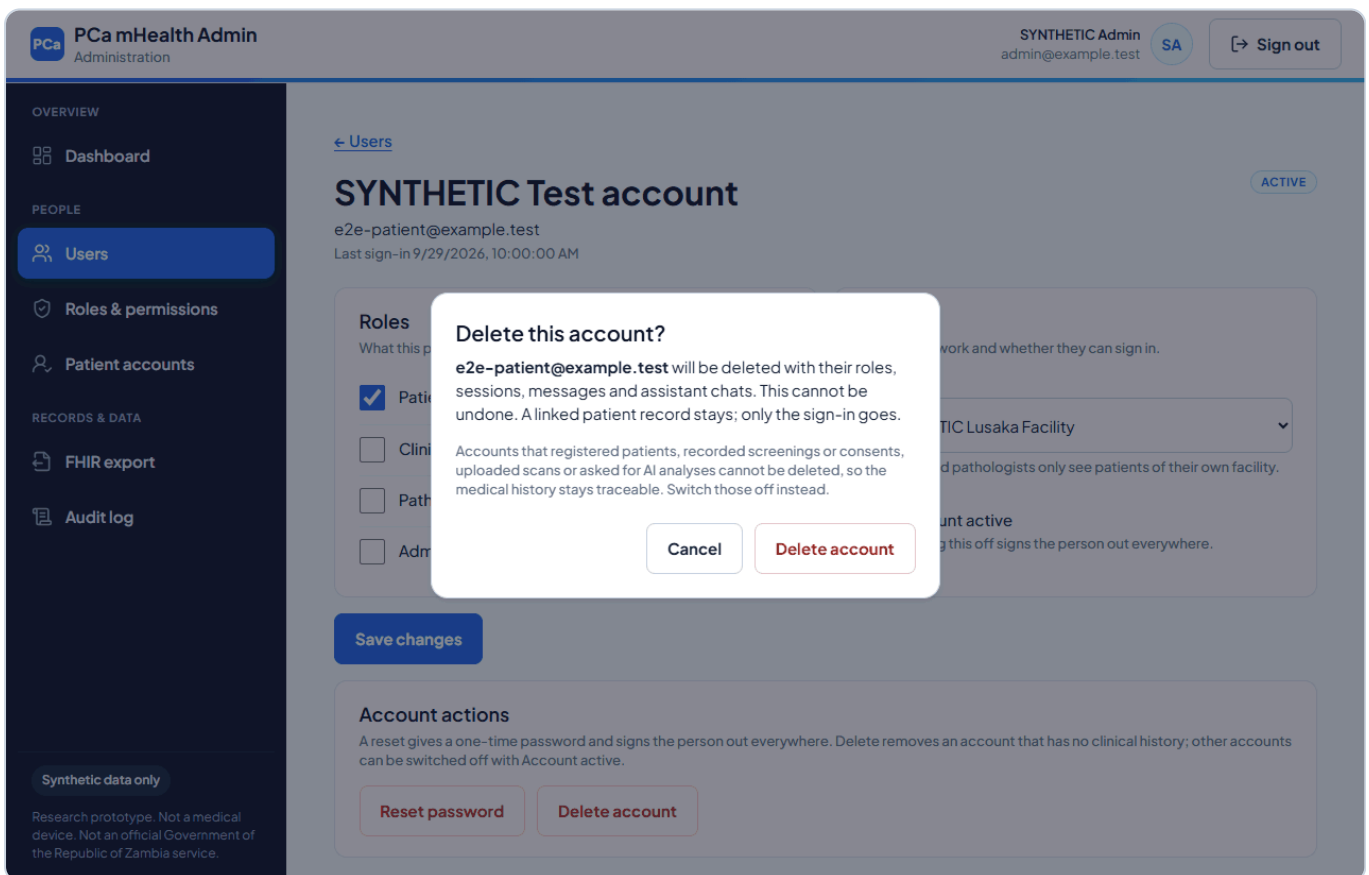
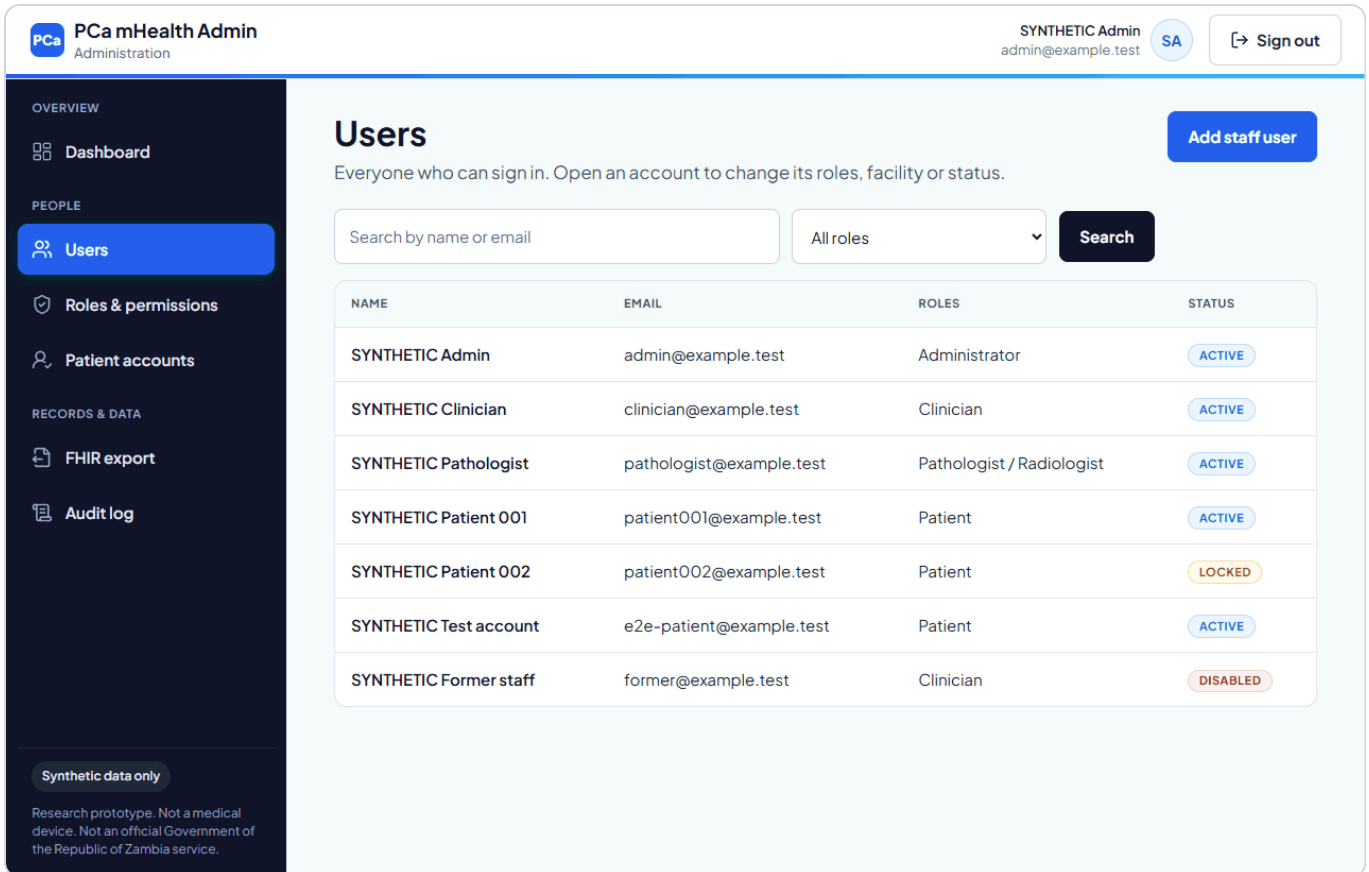


Latest from the phone app

- Added a screening record
clinician@example.test
2 min ago
- Was refused access
patient001@example.test · refused
3 min ago
- Uploaded a scan
clinician@example.test
20 min ago
- Signed in
pathologist@example.test
2 h ago
- Recorded a consent
clinician@example.test
5 h ago

Synthetic data only

Research prototype. Not a medical device. Not an official Government of the Republic of Zambia service.



These pictures show the real website, but the API's answers were replaced with synthetic ones while they were taken: the accounts are not real, and the dashboard's numbers were made up for the picture.

The dialog in that picture is this code in `src/pages/UsersPages.tsx` :

1-presentation-layer/admin-panel-web/src/pages/UsersPages.tsx

lines 405–438

```

405     {confirmDelete && (
406       <Dialog
407         title="Delete this account?"
408         onClose={() => setConfirmDelete(false)}
409         actions={
410           <>
411             <button onClick={() => setConfirmDelete(false)}>Cancel</button>
412             <button
413               className="danger"
414               disabled={remove.isPending}
415               onClick={() => remove.mutate()}
416             >
417               {remove.isPending ? 'Deleting...' : 'Delete account'}
418             </button>
419           </>
420         }
421       >
422         <p>
423           <strong>{user.email}</strong> will be deleted with their roles,
424           sessions, messages and assistant chats. This cannot be undone. A
425           linked patient record stays; only the sign-in goes.
426         </p>
427         <p className="muted">
428           Accounts that registered patients, recorded screenings or consents,
429           uploaded scans or asked for AI analyses cannot be deleted, so the
430           medical history stays traceable. Switch those off instead.
431         </p>
432       </Dialog>
433     )}
434     {temp && (
435       <TemporaryPasswordDialog
436         email={user.email}
437         password={temp}
438         onClose={() => setTemp(null)}

```

Commands (in 1-presentation-layer\admin-panel-web):

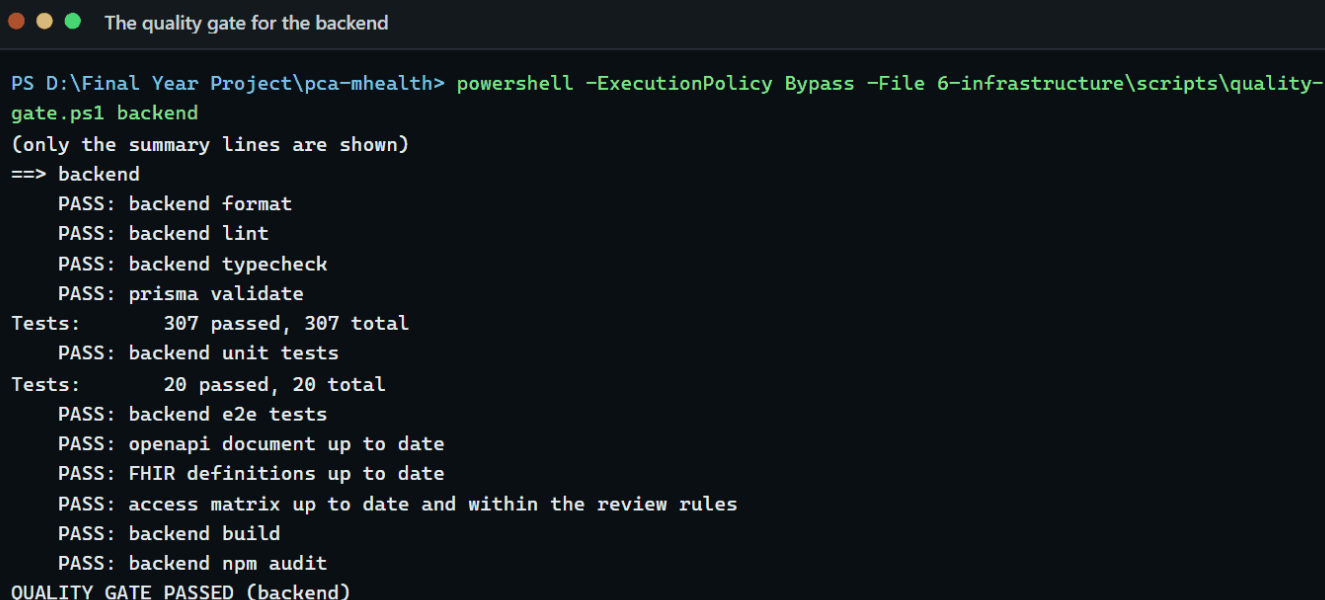
Task	Command
Run for development	npm run dev
Tests	npm test
Tests with coverage	npm run test:cov
Lint, types	npm run lint , npm run typecheck
Build the files for a web server	npm run build (output in dist/)

12. Tests and the quality gate

One script checks a whole part: formatting, lint, types, tests, build and a scan of the packages for known problems.

```
$gate = "6-infrastructure\scripts\quality-gate.ps1"
powershell -ExecutionPolicy Bypass -File $gate backend
powershell -ExecutionPolicy Bypass -File $gate mobile
powershell -ExecutionPolicy Bypass -File $gate admin-web
powershell -ExecutionPolicy Bypass -File $gate ai
# The databases' tests (needs the databases and Docker):
powershell -ExecutionPolicy Bypass -File $gate db
# Every link in the docs works; are the manual's pictures current?
powershell -ExecutionPolicy Bypass -File $gate docs
# No password or key in Git:
powershell -ExecutionPolicy Bypass -File $gate secrets
powershell -ExecutionPolicy Bypass -File $gate all
```

quality-gate.ps1 runs the real script, quality-gate.sh, with the bash that comes with Git for Windows. In **Git Bash** you can run that directly: `bash 6-infrastructure/scripts/quality-gate.sh backend`. Do not type `bash ...` in PowerShell: on this PC it starts Windows' WSL, not Git Bash.



```
PS D:\Final Year Project\pca-mhealth> powershell -ExecutionPolicy Bypass -File 6-infrastructure\scripts\quality-gate.ps1 backend
(Only the summary lines are shown)
==> backend
    PASS: backend format
    PASS: backend lint
    PASS: backend typecheck
    PASS: prisma validate
Tests:      307 passed, 307 total
    PASS: backend unit tests
Tests:      20 passed, 20 total
    PASS: backend e2e tests
    PASS: openapi document up to date
    PASS: FHIR definitions up to date
    PASS: access matrix up to date and within the review rules
    PASS: backend build
    PASS: backend npm audit
QUALITY GATE PASSED (backend)
```

Where things stood on 1 October 2026:

Part	Tests
Backend unit tests	307
Backend end-to-end tests	20
Backend database tests	230
AI service	96, and 1 skipped (it would call the real Claude); without the MedCPT models, 3 more are skipped
Phone app	280 (line coverage 90.9%)

Part	Tests
Admin website	81 (coverage 87.8%)

The rule the project has followed: no commit unless the gate for the changed part passes. If a test fails, fix the code or, if the behaviour really should change, change the test and say why in the commit.

13. Git and GitHub

- The code is in a **private** GitHub repository, `deedeestephen/final-year-project`, branch `main`.
- The history is one commit per finished piece of work, with a message that says what and why.

```
git status          # what changed
git diff            # the changes, line by line
git add -A          # stage everything
git commit -m "What changed and why"
git push origin main # send to GitHub
git log --oneline -10 # the last ten commits
git pull origin main # get changes from GitHub
```

Never commit `.env`, passwords, keys or real patient data. Before committing, run `powershell - ExecutionPolicy Bypass -File 6-infrastructure\scripts\quality-gate.ps1 secrets`.

14. When something goes wrong

What you see	Likely cause	What to do
<code>dev-up.ps1</code> : <i>The .env file is missing</i>	new folder or new PC	<code>node 6-infrastructure\scripts\gen-keys.mjs --init-env</code> , then set the database passwords
<code>dev-up.ps1</code> : <i>Database migration failed</i>	PostgreSQL is stopped, or <code>DATABASE_URL</code> is wrong	<code>Get-Service postgresql-x64-18</code> ; check the port (5432) and password in <code>.env</code>
<code>dev-up.ps1</code> : <i>docker compose failed</i>	Docker Desktop is not running	open Docker Desktop, wait for "Engine running", run again
Backend window: <code>EADDRINUSE :::3000</code>	an old backend still runs	run <code>dev-down.ps1</code> , then <code>dev-up.ps1</code>
Backend window: MongoDB Authentication failed	<code>MONGO_URL</code> and the <code>pca</code> password differ	see the troubleshooting table in local-databases.md
Admin website: <i>Email or password is incorrect</i>	wrong password, or it was changed	use <code>SEED_DEMO_PASSWORD</code> from <code>.env</code> (not <code>.env.example</code>), or <code>dev-up.ps1 -ResetDemoPasswords</code>
<i>This account is temporarily locked</i>	5 wrong tries	wait 15 minutes, or <code>dev-up.ps1 -ResetDemoPasswords</code>

What you see	Likely cause	What to do
Admin website: <i>Cannot reach the server</i>	the backend is not running	check <code>http://localhost:3000/api/v1/health</code>
App: <i>Cannot reach the server</i> on the emulator	the backend is not running, or the wrong address	the address must be <code>http://10.0.2.2:3000</code> on the emulator
App shows a white screen after the PC restarted	a debug build needs <code>flutter run</code>	run <code>flutter run</code> again (section 10.2)
Chat: <i>The assistant is not available right now</i>	the AI service is not running	start it (section 3.4) and check <code>http://127.0.0.1:8000/v1/health</code>
Chat always answers <i>No reviewed information</i>	the question is outside the knowledge base	expected; add content (section 9.4)
Voice: <i>I did not hear anything</i> on the emulator	the emulator's microphone is off	emulator › : › Microphone › <i>Virtual microphone uses host audio input</i> , or use a real phone
A test run ends at once with exit code 127 and no output	the PC ran out of memory	close Android Studio or the emulator, run <code>cd android; .\gradlew --stop</code> , try again
Docker prints <i>The "... variable is not set</i>	a <code>.env</code> value contains <code>\$</code> without quotes	put the value in single quotes
pgAdmin does not open	it must be started from the Start menu	Start › PostgreSQL 18 › pgAdmin 4

Where the messages are - Backend: its PowerShell window. - AI service: its PowerShell window. - Admin website: the browser's console (press F12). - App: the `flutter run` window. - Docker: `docker compose ... logs <service>`. - What happened in the system: the **Audit log** page on the admin website.

15. Before real patients

The prototype runs on synthetic data. These are still needed before any real use, and none of them is a programming task alone:

- **Trained and evaluated AI models.** The mocks stay labelled until then.
- **A clinician's sign-off** of the knowledge base, and **human-verified Bemba and Nyanja** translations.
- **A data-protection review** under Zambia's Data Protection Act (2021) and ethics approval, covering the speech service and, if used, Claude.
- **MongoDB access control switched on**, and a proper server with HTTPS, backups and monitoring. The list is in [security-review.md](#), items R-1 to R-8.

16. Where to read more

Question	Read
How do I test every feature by hand?	how-to-test.md
How do I run the user acceptance test with real users?	uat/README.md (only after ethics approval)
Why was it built this way?	architecture.md and decisions/
What was done, when, and what was found?	development-log.md
Which requirement is met by what, and how is it tested?	requirements-traceability.md
The databases on this PC	local-databases.md and database.md
The phone app in depth	mobile.md
Security	security.md , security-review.md , access-matrix.md
The chatbot's design	chatbot-plan.md , ADR-009 , ADR-010
Swapping the mock AI models for real ones	ai-model-integration-guide.md
Speed and load	performance.md , scalability.md

17. Updating this manual

This text is `docs/operations-manual.md` : change it in any editor. Its pictures, its PDF and its web page are made by a tool in `docs/report/tools` , from this text and from the code, so they can be made again whenever either changes:

```
$report = "docs\report\tools\report.ps1"
# What is out of date:
powershell -ExecutionPolicy Bypass -File $report check
# After changing only the text:
powershell -ExecutionPolicy Bypass -File $report pdf html
# After changing code that a picture shows:
powershell -ExecutionPolicy Bypass -File $report code images pdf html
# Everything (start dev-up.ps1 first):
powershell -ExecutionPolicy Bypass -File $report all
```

- It needs Python, Node and Chrome (or Edge). The first run installs three small Python packages.
- `docs/report/tools/contents.py` lists what each picture shows: which file and lines, which command, which phone screen.
- The quality gate's `docs` target runs `check` . It warns when a code picture shows code that has changed since the picture was taken.
- Every step, what it needs, and what to do when one stops: [report/tools/README.md](#).

- The same tool draws the **diagrams for the final-year report** (use cases, data flow, activity, sequence, state, class, database, API, architecture, timelines) from text files in `docs/report/diagrams` : `report.ps1` `diagrams` `figures` . They are listed, with captions, in [report-figures.md](#).